ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Mô hình DDPG kết hợp K Means và Cân bằng Nash trong xây dựng chiến lược Market Making tự động

...

Đinh Minh Triều
Trieu.DM216895@sis.hust.edu.vn

Chuyên ngành: Toán Tin
Chuyên sâu: Toán Ứng dụng

GVHD: TS. Nguyễn Cảnh Nam

Chữ ký

Khoa: Toán-Tin

HÀ NỘI, 06/2025

LỜI CẢM ƠN

Lời đầu tiên, em xin được gửi lời cảm ơn chân thành tới các thầy cô Khoa Toán-Tin, Đại học Bách khoa Hà Nội đã chỉ bảo tận tình, chu đáo để em xây dựng được nền tảng kiến thức phục vụ trong việc thực hiện đồ án.

Đặc biệt, em xin chân thành cảm ơn TS. Nguyễn Cảnh Nam, người đã định hướng giúp đỡ, trực tiếp hướng dẫn và tận tình chỉ bảo em trong suốt quá trình học hỏi, xây dựng và hoàn thiện đồ án này.

Em cũng xin được cảm ơn tới gia đình, người thân, các thầy cô và bạn bè thường xuyên quan tâm, động viên, chia sẻ kinh nghiệm, truyền cảm hứng, đam mê Toán học cũng cung cấp các tài liệu hữu ích trong thời gian học tập, xây dựng cũng như trong suốt quá trình thực hiện đồ án. Đây là động lực giúp em phấn đấu để hoàn thành công việc, cũng như là động lực để em hoàn thiện, phát triển bản thân hơn. Em xin chân thành cảm ơn!

Hà Nội, ngày 12 tháng 06 năm 2025

Tác giả đồ án

Đinh Minh Triều

TÓM TẮT

Trong lĩnh vực giao dịch tần suất cao (High-Frequency Trading – HFT), các nhà tạo lập thị trường (Market Maker) đóng vai trò quan trọng trong việc cung cấp thanh khoản cho thị trường. Tuy nhiên, việc tối ưu hóa chiến lược đặt lệnh trong môi trường cạnh tranh khốc liệt, với nhiều tác nhân khác nhau cùng tương tác phức tạp, vẫn là một thách thức lớn. Các phương pháp truyền thống như đặt lệnh với spread cố định hay tối ưu dựa trên mô hình xác suất thường giả định đơn giản hóa môi trường, không tính đến hành vi tương tác giữa các Market Maker khác. Một số hướng tiếp cận gần đây như sử dụng Reinforcement Learning (RL) đã cho thấy tiềm năng trong việc học chiến lược đặt lệnh linh hoạt theo dữ liệu. Tuy nhiên, phần lớn các mô hình RL hiện tại vẫn chưa khai thác yếu tố chiến lược của các tác nhân khác, hoặc thiếu tính ổn định trong môi trường thị trường thực.

Đồ án này lựa chọn hướng kết hợp giữa Game Theory và Reinforcement Learning nhằm xây dựng một mô hình Market Making thông minh hơn. Game Theory được sử dụng để mô hình hóa trò chơi giữa các Market Maker nhằm xác định trạng thái cân bằng Nash, từ đó phản ánh chiến lược tối ưu tương đối trong bối cảnh đa tác nhân. Trên cơ sở đó, agent RL sẽ học cách ra quyết định trong môi trường được cập nhật liên tục theo trạng thái chiến lược của các đối thủ, giúp tăng tính thích ứng và ổn định.

Giải pháp được triển khai theo một pipeline gồm: thu thập dữ liệu order book từ Binance, phân tích hành vi các bot thông qua clustering và nhận diện chiến lược, tính toán payoff và Nash equilibrium giữa các chiến lược, từ đó huấn luyện agent DDPG theo phương pháp online để tối ưu hóa hành động theo môi trường thực.

Đóng góp chính của đồ án là xây dựng một môi trường RL có khả năng mô phỏng sự tương tác chiến lược giữa các Market Maker khác nhau, đồng thời đề xuất cơ chế Reward mới dựa trên Nash payoff. Kết quả thực nghiệm cho thấy agent học được chiến lược Market Making có hiệu suất vượt trội so với các baseline truyền thống, thể hiện qua lợi nhuận (PnL) và chỉ số Sharpe cao hơn khi đánh giá trên dữ liệu thực tế.

Sinh viên

Mục Lục

MỞ ĐẦU **1**

CHƯƠNG 1. TỔNG QUAN NGHIÊN CỨU LIÊN QUAN **6**

1.1 Phạm vi nghiên cứu 6

1.1.1 Market Making và đặc điểm môi trường giao dịch hiện đại 6

1.1.2 Bài toán đặt ra 7

1.1.3 Phân tích đặc thù bài toán 7

1.1.4 Ứng dụng Reinforcement Learning 7

1.1.5 Tích hợp Game Theory vào chiến lược RL 8

1.1.6 Phạm vi và giới hạn của nghiên cứu 8

1.2 Nghiên cứu liên quan 9

1.2.1 Ứng dụng Reinforcement Learning trong giao dịch tài chính 9

1.2.2 Các nghiên cứu với kiến trúc mạng học sâu 9

1.2.3 Các yếu tố bổ trợ và chiến lược kết hợp 10

1.2.4 Tổng quan ưu – nhược điểm của các nghiên cứu hiện tại 10

1.2.5 Khoảng trống nghiên cứu và động lực đề tài 10

CHƯƠNG 2. KIẾN THỨC NỀN TẢNG **12**

2.1 Tổng quan về công cụ tài chính phái sinh và vai trò trong thị trường hiện đại 12

2.1.1 Thị trường phái sinh giao dịch tập trung và sự phát triển của giao dịch điện tử 13

2.1.2 Hợp đồng kỳ hạn (Forward Contracts) 13

2.1.3 Hợp đồng tương lai (Futures Contracts) 14

2.1.4 Options (Hợp đồng quyền chọn) 15

2.1.5 Types of Traders (Các loại nhà giao dịch) 16

2.1.6 Phương trình Black-Scholes-Merton 16

2.2 Lý thuyết Trò chơi và tiềm năng ứng dụng trong Bài toán Market Making.. 19

2.2.1 Trò chơi Dạng chuẩn tắc và Cân bằng Nash 19

2.3 Thuật toán K-Means trong phân cụm dữ liệu 24

2.3.1 Bài toán tối ưu tổng sai số bình phương (SSE) 24

2.3.2 Mô tả thuật toán K-Means..... 25

2.3.3 Đặc điểm và đánh giá 25

2.3.4 Các cải tiến và mở rộng 25

2.4 Thuật toán Deep Deterministic Policy Gradient (DDPG)..... 27

2.4.1 Cơ sở lý thuyết của thuật toán DDPG 27

2.4.2 Cấu trúc tổng thể..... 29

2.4.3 Thuật toán chi tiết 30

2.4.4 Các thành phần quan trọng..... 31

CHƯƠNG 3. PHƯƠNG PHÁP LUẬN NGHIÊN CỨU 32

3.1 Tổng quan 32

3.2 Thu thập dữ liệu thị trường 34

3.2.1 Mục tiêu..... 34

3.2.2 Kiến trúc hệ thống thu thập dữ liệu 34

3.2.3 Thuyết minh các bước thực hiện 34

3.2.4 Minh hoạ dữ liệu đầu ra 36

3.2.5 Đảm bảo tính liên tục và độ tin cậy 37

3.3 Phân cụm các lệnh (Order Clustering) 37

3.4 Nhận diện chiến lược từng bot..... 42

3.4.1 Mục tiêu..... 42

3.4.2 Tóm tắt đặc trưng của mỗi cụm..... 42

3.4.3 Gán nhãn loại bot..... 42

3.4.4 Nhận diện chiến lược cụ thể 43

3.4.5 Thuật toán nhận diện chiến lược 43
3.5 Xây dựng ma trận Payoff và tìm Cân bằng Nash 43
3.5.1 Xây dựng hàm Payoff 44
3.5.2 Sinh không gian chiến lược 45
3.5.3 Tìm chiến lược phản hồi tốt nhất 45
3.5.4 Thuật toán Best Response Dynamics..... 45
3.6 Quá trình huấn luyện tác nhân DDPG trong môi trường Market Making 47
3.6.1 Sinh quan sát (Observation) 47
3.6.2 Tác nhân chọn hành động (Action Selection) 48
3.6.3 Mô phỏng thị trường và tính toán phần thưởng (Reward Computation) 48
3.6.4 Cập nhật mô hình và replay buffer 49
3.6.5 Học tăng cường online liên tục (Online RL Training) 49
3.7 Huấn luyện mô hình Reinforcement Learning cho Market Making 49
3.7.1 Tải dữ liệu order book thực tế từ file Parquet 50
3.7.2 Khởi tạo môi trường RL: `MarketMakingEnv`..... 50
3.7.3 Thuật toán huấn luyện DDPG..... 51
3.7.4 Tính phần thưởng từ chiến lược Nash 51
3.7.5 Lưu mô hình định kỳ 52
3.8 Đánh giá chiến lược RL bằng mô phỏng khớp lệnh 53
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM 57
4.1 Các tham số đánh giá 57
4.1.1 Profit and Loss (PnL) 57
4.1.2 Sharpe Ratio..... 58
4.2 Phương pháp mô phỏng..... 58
4.2.1 Các baseline được sử dụng để so sánh 60

4.2.2 Thiết kế thí nghiệm và tiêu chí đánh giá 60

4.3 Kết quả Backtest trên dữ liệu kiểm thử..... 61

4.4 Nhận xét, giải thích kết quả, so sánh với các baseline 68

KẾT LUẬN 71

REFERENCE 73

DANH SÁCH HÌNH ẢNH

Hình 3.1 Quy trình thu thập và lưu trữ dữ liệu từ Binance 34
Hình 3.2 Pipeline phân cụm các lệnh theo hành vi 38
Hình 3.3 Ví dụ minh hoạ phân cụm lệnh trên Orderbook Snapshot . . . 40

Hình 4.1 Biểu đồ PnL trên Bộ dữ liệu Test 1 61
Hình 4.2 Biểu đồ PnL trên Bộ dữ liệu Test 2 62
Hình 4.3 Biểu đồ PnL trên Bộ dữ liệu Test 3 63
Hình 4.4 Biểu đồ PnL trên Bộ dữ liệu Test 4 64
Hình 4.5 Biểu đồ PnL trên Bộ dữ liệu Test 5 65
Hình 4.6 Biểu đồ PnL trên Bộ dữ liệu Test 6 66
Hình 4.7 Biểu đồ PnL trên Bộ dữ liệu Test 7 67
Hình 4.8 Biểu đồ PnL trên Bộ dữ liệu Test 8 68

DANH SÁCH BẢNG

Bảng 2.1 Ví dụ 1: Song đề Tù nhân 20

Bảng 3.1 Danh sách các đặc trưng hành vi được sử dụng 40
Bảng 3.2 Tổng quan pipeline đánh giá chiến lược RL 56

Bảng 4.1 Kết quả Sharpe Ratio của mô hình đề xuất trên từng tập dữ liệu 68
Bảng 4.2 So sánh hiệu suất Sharpe Ratio giữa các mô hình 69

MỞ ĐẦU

Đặt vấn đề

Trong những năm gần đây, thị trường tiền mã hóa (Cryptocurrency) đã phát triển nhanh chóng cả về quy mô và độ sâu thanh khoản. Trên các sàn giao dịch như Binance, hoạt động mua bán diễn ra liên tục với khối lượng giao dịch cực lớn mỗi giây. Tuy nhiên, không phải nhà đầu tư cá nhân hay tổ chức nào cũng trực tiếp đặt lệnh theo cách thủ công. Trên thực tế, phần lớn các lệnh giao dịch hiện nay được tạo ra và xử lý bởi các thuật toán giao dịch tự động (Algorithmic Trading Bots).

Một trong những dạng phổ biến nhất của các thuật toán này là Market Making Bot — các bot tự động đặt lệnh mua và bán ở hai phía của sổ lệnh với mục tiêu tạo lập thanh khoản, hưởng chênh lệch giá (spread), và duy trì vị thế ổn định. Những bot này đóng vai trò quan trọng trong việc giữ cho thị trường vận hành trơn tru, giúp giảm độ trượt giá và cải thiện khả năng khớp lệnh.

Tuy nhiên, trong môi trường ngày càng cạnh tranh của các sàn giao dịch, các bot giao dịch không còn đơn độc. Thị trường hiện nay là một hệ sinh thái đa tác nhân, nơi nhiều loại bot với chiến lược khác nhau cùng tồn tại và cạnh tranh trực tiếp. Các chiến lược này bao gồm:

- Market Maker (MM): tập trung vào việc duy trì thanh khoản quanh mức giá trung tâm.
- Trend Follower (TF): phản ứng theo xu hướng giá lên hoặc xuống.
- Arbitrage Trader (AT): khai thác sự chênh lệch giá giữa các thị trường.

Trong bối cảnh đó, mỗi bot hoạt động không chỉ dựa trên trạng thái thị trường, mà còn chịu ảnh hưởng bởi hành vi của các bot khác. Điều này dẫn đến một bài toán giao dịch tự động phức tạp, nơi các tác nhân tương tác chiến lược với nhau trong thời gian thực. Việc xây dựng một thuật toán giao dịch hiệu quả không chỉ đòi hỏi năng lực dự đoán diễn biến thị trường, mà còn cần khả năng thích ứng với các đối thủ, phản ứng với sự thay đổi liên tục trong sổ lệnh (order book) và cấu trúc thị trường.

Từ thực tiễn trên, bài toán đặt ra là:

Làm thế nào để xây dựng một thuật toán giao dịch tự động dạng Market Making có khả năng đưa ra hành động tối ưu trong môi trường thị trường có nhiều bot cạnh tranh, với các chiến lược khác nhau, và trạng thái thị trường biến đổi liên tục theo thời gian?

Bài toán này đặt ra nhiều thách thức, bao gồm:

- Hiểu và mô hình hóa thị trường: cần có cơ chế thu thập và phân tích dữ liệu order book có độ phân giải cao để trích xuất thông tin hữu ích phục vụ ra quyết định.
- Phân biệt các loại hành vi giao dịch: vì nhiều loại bot với chiến lược khác nhau cùng tham gia thị trường, nên thuật toán cần khả năng phân loại hoặc phản ứng phù hợp với từng đối thủ.
- Đưa ra quyết định trong môi trường động: thị trường thay đổi nhanh, nên thuật toán phải phản ứng trong thời gian thực với thông tin mới liên tục.

Việc giải quyết bài toán này có ý nghĩa lớn cả về mặt học thuật lẫn ứng dụng thực tế:

- Về học thuật: đây là một bài toán thuộc lĩnh vực liên ngành giữa tài chính định lượng, học máy, và phân tích dữ liệu thời gian thực, có tiềm năng nghiên cứu sâu rộng.
- Về ứng dụng: xây dựng được một thuật toán Market Making thông minh có thể giúp cải thiện hiệu quả giao dịch, giảm rủi ro, và tạo ra lợi thế cạnh tranh lớn cho các tổ chức giao dịch thuật toán trong thị trường crypto.

Bối cảnh và vấn đề nghiên cứu

Tổng quan các nghiên cứu hiện tại

Nghiên cứu của Nevmyvaka et al. (2006) [1] là một trong những công trình đầu tiên áp dụng RL trên quy mô lớn cho bài toán tối ưu thực hiện lệnh (trade execution). Nhóm tác giả thiết kế một thuật toán RL kết hợp Q-learning và lập trình động, tận dụng cấu trúc đặc thù của sổ lệnh. Kết quả cho thấy chiến lược học được cải thiện hiệu suất tới 50% so với các phương pháp đơn giản. Họ cũng phân tích sâu các biến vi mô (microstructure) như chi phí đặt lệnh thị trường và cách chúng ảnh hưởng đến chiến lược.

Nghiên cứu của Yasin và Gill (2024) [2] tập trung vào việc xây dựng khung RL cho giao dịch định lượng với trọng tâm vào khai thác chỉ báo kỹ thuật. Họ chỉ ra rằng nhiều nghiên cứu hiện tại chỉ dừng ở kiểm nghiệm trên dữ liệu quá khứ, thiếu kiểm chứng thực tiễn và chưa tận dụng đầy đủ các chỉ báo tài chính.

Cuối cùng, Sarkar (2023) [3] sử dụng Deep Q-Network (DQN) để học chính sách giao dịch tối ưu từ dữ liệu lịch sử giá cổ phiếu và chỉ báo kỹ thuật. Mô hình cho kết quả tích cực khi so sánh với các chiến lược truyền thống như SMA và Buy-and-Hold, gợi ý tiềm năng lớn của RL trong tự động hóa giao dịch.

Hạn chế của các giải pháp hiện tại

- Phần lớn nghiên cứu chỉ dừng lại ở mô phỏng với dữ liệu lịch sử, chưa được kiểm nghiệm trong môi trường thị trường thực.
- Thiếu sự kết hợp giữa các yếu tố vi mô của thị trường (microstructure) với hành vi chiến lược của các tác nhân khác.
- Các chiến lược học được thường khó giải thích hoặc chưa gắn chặt với lý thuyết trò chơi, hạn chế khả năng phân tích sâu hành vi thị trường.

Mục tiêu nghiên cứu và khung lý thuyết

Mục tiêu nghiên cứu

Mục tiêu chính của đồ án là xây dựng một hệ thống giao dịch tự động dạng Market Making, có khả năng:

- Học và thích nghi với môi trường thị trường biến động cao.
- Phân tích hành vi chiến lược của các bot khác (Market Maker, Trend Follower, Arbitrage Trader, v.v).
- Tối ưu hóa chiến lược đặt lệnh thông qua Reinforcement Learning (RL) có kết hợp Game Theory.
- Áp dụng hệ thống vào dữ liệu thực từ sổ lệnh (order book) của sàn Binance, theo thời gian thực.

Hệ thống hướng đến việc mô hình hóa thị trường như một trò chơi giữa các bot, từ đó tối ưu hành động của bot Market Making cá nhân dựa trên chiến lược cân bằng (Nash Equilibrium) và dữ liệu thị trường.

Khung giải pháp đề xuất

- Giải quyết tính chiến lược của thị trường: Mô hình hóa thị trường như trò chơi không hợp tác giữa nhiều loại bot, sử dụng thuật toán clustering để phân loại hành vi từ order book.
- Nhận diện chiến lược bot đối thủ: Phân tích đặc trưng lệnh (spread, volume, vị trí, thời gian tồn tại,...) để xác định chiến lược con của từng bot.
- Tối ưu phần thưởng RL theo lý thuyết trò chơi: Tính payoff giữa các chiến lược, tìm Nash Equilibrium và dùng nó để xác định reward lý thuyết cho agent.
- Thích nghi liên tục với thị trường: Thiết kế môi trường RL dạng online, cập nhật clustering, chiến lược và Nash sau mỗi bước.
- Tăng cường quan sát cho agent: Mở rộng observation lên 7 chiều, bao gồm

thông tin agent, thị trường, hành vi bot khác và reward lịch sử.

Đóng góp của đồ án

Đồ án này có 3 đóng góp chính như sau:

- Đề xuất một pipeline phân tích hành vi thị trường từ dữ liệu order book theo thời gian thực, bao gồm phân cụm bot, nhận diện chiến lược và xây dựng ma trận payoff.
- Kết hợp Lý thuyết Trò chơi (Game Theory) vào môi trường Reinforcement Learning thông qua reward được tính từ Nash Equilibrium giữa các chiến lược bot.
- Xây dựng môi trường Market Making dạng real-time RL với không gian quan sát mở rộng, cho phép agent học và thích nghi liên tục với thị trường biến động.

Cấu trúc đồ án

Cấu trúc của báo cáo đồ án tốt nghiệp này được tổ chức như sau:

Chương 1 trình bày tổng quan các nghiên cứu liên quan nhằm xây dựng nền tảng lý thuyết và xác định khoảng trống trong lĩnh vực. Nội dung bao gồm vai trò của chiến lược Market Making trong thị trường hiện đại, đặc điểm môi trường giao dịch tài chính điện tử, và bài toán cụ thể mà đề tài hướng đến. Chương cũng giới thiệu tổng quan về Reinforcement Learning, lý do kết hợp với Game Theory, và phân tích các công trình trước đây ứng dụng RL trong giao dịch. Cuối cùng, chương chỉ ra khoảng trống nghiên cứu hiện tại, làm rõ động lực và đóng góp của đề tài.

Chương 2 trình bày các kiến thức nền tảng phục vụ cho phương pháp nghiên cứu, bao gồm các lý thuyết và thuật toán chính được sử dụng trong đồ án. Nội dung gồm: giới thiệu công cụ tài chính phái sinh và vai trò trong Market Making, tổng quan Lý thuyết Trò chơi với mô hình chuẩn tắc và cân bằng Nash, thuật toán K-Means trong phân cụm hành vi bot, và chi tiết thuật toán Deep Deterministic Policy Gradient (DDPG) cho không gian hành động liên tục. Chương này là cơ sở lý thuyết cho quá trình xây dựng và huấn luyện mô hình sau này.

Chương 3 trình bày quy trình và phương pháp luận giải quyết bài toán Market Making bằng Reinforcement Learning kết hợp Game Theory. Nội dung bao gồm: thu thập dữ liệu order book thực từ Binance qua WebSocket API, phân cụm lệnh để nhận diện nhóm hành vi bot và gán nhãn chiến lược, xây dựng ma trận payoff và tìm gần đúng cân bằng Nash bằng thuật toán Best Response Dynamics. Cuối chương là quá trình xây dựng môi trường học tăng cường MarketMakingEnv, nơi tác nhân DDPG tương tác, nhận quan sát, chọn hành động, tính reward dựa trên

khoảng cách tới chiến lược Nash, và được huấn luyện/backtest với dữ liệu thực. Đây là chương trọng tâm thể hiện toàn bộ đóng góp kỹ thuật của đề tài.

Cuối cùng, Chương 4 trình bày các kết quả thực nghiệm, bao gồm mô phỏng khớp lệnh để đánh giá chiến lược của tác nhân RL trên dữ liệu thị trường thực, với các chỉ số đánh giá như lợi nhuận (PnL) và hệ số Sharpe Ratio.

CHƯƠNG 1. TỔNG QUAN NGHIÊN CỨU LIÊN QUAN

Chương này trình bày tổng quan các công trình nghiên cứu có liên quan đến đề tài, giúp đặt nền móng lý thuyết và xác định khoảng trống trong nghiên cứu hiện tại. Trong chương này, tác giả đi từ việc mô tả khái quát vai trò của chiến lược Market Making trong thị trường hiện đại, các đặc điểm của môi trường giao dịch tài chính điện tử, cho đến việc đặt ra bài toán cụ thể cần giải quyết. Các nội dung quan trọng bao gồm phân tích đặc thù của môi trường giao dịch, giới thiệu tổng quan về Reinforcement Learning và lý do chọn hướng tiếp cận kết hợp Game Theory. Sau đó, chương cung cấp cái nhìn hệ thống về các công trình trước đây ứng dụng Reinforcement Learning trong giao dịch, bao gồm cả các kiến trúc học sâu, chiến lược kết hợp và đánh giá các ưu nhược điểm của từng hướng tiếp cận. Cuối chương là phần phân tích khoảng trống nghiên cứu hiện tại, từ đó làm rõ động lực và đóng góp tiềm năng của đề tài.

1.1 Phạm vi nghiên cứu

Trong bối cảnh thị trường tài chính ngày càng phát triển với sự tham gia mạnh mẽ của các hệ thống giao dịch tự động và các thuật toán trí tuệ nhân tạo, việc xây dựng các mô hình học tăng cường (Reinforcement Learning - RL) cho các chiến lược giao dịch như market making trở nên ngày càng cấp thiết. Thị trường tiền mã hóa (Cryptocurrency), điển hình như sàn giao dịch Binance, là một môi trường đầy biến động và cạnh tranh, nơi các nhà tạo lập thị trường (Market Maker) cạnh tranh từng mili giây để giành ưu thế thanh khoản. Trong phần này, chúng tôi sẽ trình bày bối cảnh tổng thể của bài toán Market Making, vai trò của học tăng cường và lý thuyết trò chơi (Game Theory) trong việc tối ưu chiến lược, và các giả định, giới hạn chính của phạm vi nghiên cứu này.

1.1.1 Market Making và đặc điểm môi trường giao dịch hiện đại

Market Making là quá trình trong đó một tác nhân (Market Maker) cung cấp thanh khoản cho thị trường bằng cách liên tục đặt lệnh mua (bid) và bán (ask). Lợi nhuận của Market Maker chủ yếu đến từ spread giữa giá mua và bán. Tuy nhiên, để duy trì vị thế cạnh tranh, Market Maker cần đưa ra các quyết định tối ưu về mức giá, khối lượng và thời điểm đặt lệnh trong khi đối mặt với các rủi ro như biến động giá, mất cân bằng inventory, và cạnh tranh với các Market Maker khác.

Đặc biệt, trên thị trường tài sản số như Binance, các Market Maker hoạt động với tần suất cao (HFT - High-Frequency Trading), và hành vi của họ thường ẩn dưới các mẫu lệnh phức tạp. Do đó, việc mô hình hóa một Market Maker hiệu quả cần không chỉ dựa vào các chỉ báo kỹ thuật truyền thống mà còn tích hợp các phương

pháp học tự động, nhằm thích ứng liên tục với thị trường và đối thủ.

1.1.2 Bài toán đặt ra

Bài toán đặt ra trong nghiên cứu này là, làm thế nào để một tác nhân Market Maker có thể học được một chiến lược tối ưu nhằm:

- Tối đa hóa lợi nhuận kỳ vọng,
- Giảm rủi ro liên quan đến inventory và biến động giá,
- Thích ứng với chiến lược của các market maker khác đang hoạt động đồng thời trên thị trường.

Bài toán này có thể được xem như một trò chơi lặp nhiều bước (Repeated Stochastic Game), trong đó mỗi agent (bao gồm tác nhân của ta và các bot khác) thực hiện hành động theo chiến lược của mình, nhận thưởng (payoff), và điều chỉnh chiến lược theo thời gian.

1.1.3 Phân tích đặc thù bài toán

Một số đặc điểm riêng biệt của bài toán Market Making trong nghiên cứu này bao gồm:

- **Không gian hành động liên tục**: Market Maker có thể chọn mức spread và volume bất kỳ trong một phạm vi liên tục, không giới hạn ở các mức rời rạc.
- **Tác nhân không tác động đến thị trường**: Giả định được đặt ra là tác nhân học (agent) có kích thước giao dịch nhỏ và không ảnh hưởng đến trạng thái order book.
- **Thông tin quan sát bị giới hạn**: Tác nhân không biết chính xác hành vi các bot khác mà chỉ quan sát được order book snapshot.
- **Thị trường động và phi tuyến**: Hành vi thị trường thay đổi liên tục và khó dự đoán, nên yêu cầu agent có khả năng thích nghi linh hoạt.

Những yếu tố trên khiến bài toán trở nên phức tạp và không thể giải quyết tốt bằng các phương pháp truyền thống như rule-based hay chiến lược tĩnh.

1.1.4 Ứng dụng Reinforcement Learning

Reinforcement Learning cung cấp một khuôn khổ tự nhiên để giải quyết bài toán trên. Một tác nhân RL sẽ học cách tương tác với môi trường bằng cách thử nghiệm nhiều chiến lược, nhận về phần thưởng và điều chỉnh chính sách hành động theo thời gian. Các lợi thế của RL trong bối cảnh này bao gồm:

- Tự động khám phá chiến lược tối ưu thông qua trải nghiệm.
- Tối ưu hóa phần thưởng dài hạn thay vì chỉ lợi nhuận tức thời.

- Khả năng thích nghi với thay đổi trong hành vi thị trường và bot khác.

Trong phạm vi đề tài này, tác nhân RL sẽ được huấn luyện với thuật toán Deep Deterministic Policy Gradient (DDPG), cho phép hành động trong không gian liên tục (liên quan đến spread và volume), đồng thời cập nhật chính sách theo dữ liệu thực từ order book Binance.

1.1.5 Tích hợp Game Theory vào chiến lược RL

Để mô hình hóa và phân tích hành vi các Market Maker khác, nghiên cứu này ứng dụng thêm lý thuyết trò chơi. Cụ thể, sau mỗi bước huấn luyện, dữ liệu order book sẽ được phân cụm thành các nhóm hành vi (representing bots), nhận diện chiến lược, và xây dựng ma trận payoff giữa các chiến lược. Bằng cách sử dụng phương pháp phản hồi tốt nhất (Best Response Dynamics), nghiên cứu tìm xấp xỉ điểm cân bằng Nash, qua đó đánh giá phần thưởng tiềm năng của tác nhân nếu áp dụng chiến lược tối ưu tương ứng.

Cách tiếp cận này giúp tác nhân RL không chỉ học từ kinh nghiệm của chính mình mà còn khai thác được thông tin từ môi trường chiến lược xung quanh.

1.1.6 Phạm vi và giới hạn của nghiên cứu

Phạm vi nghiên cứu của đề tài bao gồm:

- Phân tích dữ liệu order book thực từ sàn Binance,
- Áp dụng clustering để nhận diện các bot Market Making,
- Xây dựng mô hình chiến lược cho từng bot và tính toán payoff,
- Huấn luyện agent RL với DDPG sử dụng pipeline real-time (mỗi bước dùng dữ liệu mới),
- Đánh giá hiệu quả dựa trên phần thưởng Nash và quan sát hành vi chiến lược của agent.

Tuy nhiên, nghiên cứu vẫn tồn tại một số giới hạn:

- Không tính đến tác động lệnh của agent đến thị trường (agent nhỏ),
- Không đánh giá trong môi trường giao dịch thực (live trading),
- Giới hạn số lượng cluster (số bot) và không xét nhiều loại tài sản.

Kết luận

Phần này đã phân tích rõ ngữ cảnh bài toán Market Making trong môi trường tài chính hiện đại, đặc biệt là thị trường tiền mã hóa. Tác giả đã nêu bật sự phức tạp của bài toán, sự cần thiết của các phương pháp học tăng cường và vai trò hỗ trợ của lý thuyết trò chơi trong việc phân tích hành vi các tác nhân khác. Phần tiếp theo sẽ

trình bày các công trình nghiên cứu liên quan, từ đó rút ra các động lực và đóng góp riêng biệt của đề tài này.

1.2 Nghiên cứu liên quan

Trong phần này, tác giả trình bày tổng quan các nghiên cứu liên quan đến việc ứng dụng Reinforcement Learning (RL) trong lĩnh vực tài chính định lượng, đặc biệt là các bài toán giao dịch và tối ưu danh mục đầu tư. Mục tiêu của phần này là phân tích điểm mạnh, điểm yếu của các phương pháp đã có, từ đó xác định khoảng trống nghiên cứu và làm rõ động lực thực hiện của đồ án này. Bên cạnh đó, tác giả cũng giới thiệu các kiến thức nền tảng có liên quan đến các mô hình được sử dụng, nhằm cung cấp bối cảnh đầy đủ cho phần phát triển mô hình ở các chương sau.

1.2.1 Ứng dụng Reinforcement Learning trong giao dịch tài chính

Reinforcement Learning đã trở thành một công cụ được quan tâm rộng rãi trong tài chính định lượng nhờ khả năng tự học từ dữ liệu và ra quyết định trong môi trường không chắc chắn.

Moody và Saffell (2001) [4] áp dụng RL cho chiến lược giao dịch trên thị trường mô phỏng, cho kết quả vượt trội so với các chiến lược truyền thống, nhưng mô hình còn đơn giản và chưa phản ánh được sự phức tạp của thị trường thật.

Bertoluzzo và De Nicolao (2006) [5] dùng RL để tối ưu danh mục cổ phiếu và đạt hiệu suất tốt, tuy nhiên thiếu phân tích về rủi ro, chi phí giao dịch và yếu tố thị trường ngẫu nhiên.

Chen et al. (2019) [6] sử dụng Deep RL trong giao dịch cổ phiếu, khai thác tốt dữ liệu phức tạp qua mạng nơron sâu, nhưng chưa thử nghiệm trên môi trường giao dịch thực nên dễ bị overfitting.

Guo et al. (2020) [7] đề xuất mô hình DRL cho thị trường Bitcoin Futures với hiệu suất cao, phù hợp với thị trường biến động mạnh, nhưng chưa được kiểm nghiệm trên thị trường truyền thống.

Gu et al. (2021) [8] tích hợp chi phí giao dịch vào mô hình RL, giúp cải thiện hiệu suất điều chỉnh theo rủi ro, song môi trường mô phỏng vẫn chưa mô hình hóa được tương tác giữa nhiều tác nhân thị trường.

1.2.2 Các nghiên cứu với kiến trúc mạng học sâu

Một số nghiên cứu gần đây đã ứng dụng kiến trúc mạng học sâu vào giao dịch tài chính. Nghiên cứu năm 2016 về RDNN [9] sử dụng mạng hồi tiếp để ra quyết định giao dịch thời gian thực mà không cần chỉ báo kỹ thuật, nhưng bị giới hạn bởi phạm vi dữ liệu. Mô hình kết hợp DDPG và RCNN [10] tích hợp cả dữ liệu kỹ thuật và phân tích cảm xúc, cho thấy tiềm năng trong việc giảm độ bất định, tuy nhiên phần

thưởng nhị phân gây hạn chế về khả năng đo lường hiệu suất thực sự. Phương pháp EIIE [11] sử dụng tổ hợp CNN, RNN và LSTM để tối ưu danh mục, đạt hiệu suất tốt trong thị trường tiền mã hóa nhưng chưa được kiểm nghiệm trên các thị trường tài chính truyền thống.

1.2.3 Các yếu tố bổ trợ và chiến lược kết hợp

Kết hợp chiến lược kỹ thuật và RL cũng là hướng tiếp cận phổ biến. Một nghiên cứu phân tích chiến lược "double average" (trung bình kép) đã chứng minh tính hiệu quả trong việc xác định điểm mua bán tối ưu, đặc biệt trong môi trường biến động. Tuy nhiên, các chiến lược ngắn hạn thường thiếu tính ổn định và khó duy trì hiệu suất trong thời gian dài. Do đó, nhiều nghiên cứu đã chuyển hướng sang đánh giá mô hình theo lợi nhuận dài hạn hoặc hiệu suất điều chỉnh theo rủi ro.

1.2.4 Tổng quan ưu – nhược điểm của các nghiên cứu hiện tại

Ưu điểm:

- RL có khả năng học chính sách từ dữ liệu quá khứ mà không cần mô hình hóa thị trường một cách cụ thể.
- Các mô hình DRL hiện đại (DDPG, PPO, A3C) thể hiện hiệu suất vượt trội so với các chiến lược giao dịch truyền thống.
- Khả năng tích hợp nhiều loại dữ liệu đầu vào (giá, volume, cảm xúc thị trường, v.v.).

Nhược điểm:

- Hầu hết mô hình chỉ được kiểm nghiệm qua backtest, không chứng minh hiệu quả trong live trading.
- Đa phần giả định thị trường không thay đổi hoặc không xét đến ảnh hưởng qua lại giữa các tác nhân.
- Cần dữ liệu lớn, huấn luyện lâu và khó tái hiện trong môi trường thực.
- Phần thưởng và chính sách thường thiếu yếu tố quản lý rủi ro.

1.2.5 Khoảng trống nghiên cứu và động lực đề tài

Qua tổng quan trên, có thể thấy các mô hình RL trong tài chính tuy hứa hẹn nhưng còn một số hạn chế:

- Ít mô hình xem xét ảnh hưởng chiến lược của các tác nhân khác (multi-agent interaction), trong khi thị trường tài chính là một trò chơi chiến lược giữa nhiều Market Maker.
- Các nghiên cứu chưa xem xét khả năng kết hợp Game Theory vào RL để mô

hình hóa hành vi tương tác và chiến lược cân bằng (Nash Equilibrium).

- Phần lớn mô hình chưa tận dụng thông tin từ order book ở cấp độ vi mô (microstructure).

Vì vậy, đồ án này hướng đến phát triển một môi trường giao dịch mô phỏng Market Making kết hợp giữa Reinforcement Learning và Game Theory. Hệ thống sẽ phân tích dữ liệu order book thực tế để nhận diện chiến lược các bot đang hoạt động, tính toán payoff thực tế và huấn luyện một agent RL học cách phản ứng tối ưu theo các chiến lược cân bằng. Cách tiếp cận này hứa hẹn sẽ phản ánh sát hơn với thị trường thực và cải thiện khả năng tổng quát hóa của mô hình RL.

Kết luận

Chương này đã trình bày chi tiết các công trình nghiên cứu tiêu biểu trong lĩnh vực áp dụng Reinforcement Learning vào giao dịch tài chính và tối ưu danh mục. Các nghiên cứu đều chỉ ra tiềm năng lớn của RL trong việc cải thiện hiệu suất giao dịch. Tuy nhiên, vẫn tồn tại nhiều hạn chế như thiếu tính tương tác chiến lược, chưa ứng dụng vào môi trường live trading, và bỏ qua thông tin microstructure của thị trường. Những hạn chế này chính là động lực để đề tài này hướng tới xây dựng một môi trường mô phỏng RL tích hợp Game Theory và thông tin order book, nhằm học được các chính sách giao dịch hiệu quả và ổn định hơn.

CHƯƠNG 2. KIẾN THỨC NỀN TẢNG

Chương này trình bày các kiến thức nền tảng phục vụ cho phương pháp nghiên cứu được áp dụng trong đồ án. Các lý thuyết và thuật toán chủ chốt như công cụ tài chính phái sinh, Lý thuyết Trò chơi, K-Means và học tăng cường DDPG sẽ được giới thiệu một cách có hệ thống. Những nội dung này không chỉ giúp hiểu rõ bối cảnh kỹ thuật của bài toán Market Making mà còn là cơ sở lý luận để phát triển mô hình mô phỏng, nhận diện chiến lược và huấn luyện tác nhân giao dịch tự động trong các chương tiếp theo.

2.1 Tổng quan về công cụ tài chính phái sinh và vai trò trong thị trường hiện đại

Trong hơn ba thập kỷ qua, các công cụ tài chính phái sinh như hợp đồng kỳ hạn (*forward*), hợp đồng tương lai (*futures*), quyền chọn (*options*) và hoán đổi (*swaps*) đã đóng vai trò trung tâm trong hoạt động của thị trường tài chính hiện đại. Chúng được sử dụng rộng rãi bởi các tổ chức tài chính, nhà đầu tư tổ chức, quỹ phòng hộ, và các doanh nghiệp nhằm phục vụ ba mục tiêu chính: **phòng hộ rủi ro (hedging)**, **đầu cơ (speculation)** và **kinh doanh chênh lệch giá (arbitrage)**.

Giá trị danh nghĩa của thị trường phái sinh hiện nay vượt xa cả quy mô thị trường cổ phiếu toàn cầu và chiếm một phần lớn trong hệ thống tài chính. Sự phát triển nhanh chóng của thị trường này là hệ quả tất yếu của nhu cầu ngày càng cao trong việc **quản trị rủi ro tài chính** và **tăng hiệu quả sử dụng vốn**.

Tuy nhiên, các công cụ phái sinh cũng từng là tâm điểm chỉ trích, đặc biệt sau cuộc khủng hoảng tài chính toàn cầu năm 2007–2008, khi các sản phẩm phái sinh phức tạp gắn liền với nợ thế chấp rủi ro cao (*mortgage-backed securities*) góp phần gây ra tổn thất nghiêm trọng. Kể từ đó, thị trường phái sinh đã được điều tiết chặt chẽ hơn, với yêu cầu nghiêm ngặt về vốn, thanh khoản và quản lý rủi ro.

Trong bối cảnh đó, vai trò của **các nhà tạo lập thị trường (Market Makers)** ngày càng trở nên quan trọng. Họ đóng vai trò cung cấp thanh khoản liên tục, định hình giá thị trường thông qua việc chào mua và chào bán tài sản tài chính – bao gồm cả tài sản cơ sở và công cụ phái sinh. Đặc biệt trong các thị trường hiện đại như tiền điện tử hoặc các sàn giao dịch điện tử tốc độ cao, Market Maker sử dụng các chiến lược phức tạp, kết hợp giữa lý thuyết tài chính cổ điển và các kỹ thuật học máy, để tối ưu hóa hành vi đặt lệnh.

Việc hiểu rõ bản chất của các công cụ tài chính, cách chúng được định giá, sử dụng và quản lý rủi ro, là nền tảng thiết yếu để xây dựng các chiến lược Market Making

hiệu quả. Do đó, phần tiếp theo của chương này sẽ tập trung vào việc trình bày các khái niệm cơ bản trong tài chính định lượng, đặc biệt là cấu trúc của thị trường phái sinh, hành vi của các tác nhân trên thị trường, và các yếu tố ảnh hưởng đến tính thanh khoản và chi phí giao dịch. Các nội dung của phần này được chủ yếu tham khảo từ [12].

2.1.1 Thị trường phái sinh giao dịch tập trung và sự phát triển của giao dịch điện tử

Thị trường phái sinh giao dịch tập trung (Exchange-Traded Markets) là nơi các hợp đồng phái sinh tiêu chuẩn được mua bán công khai dưới sự giám sát của sàn giao dịch, với quy mô, điều kiện và thời gian đáo hạn được chuẩn hóa.

Một trong những sàn phái sinh đầu tiên là *Chicago Board of Trade (CBOT)*, thành lập năm 1848 để chuẩn hóa việc trao đổi ngũ cốc, về sau phát triển thành các hợp đồng tương lai. Đến năm 1919, *Chicago Mercantile Exchange (CME)* ra đời và hiện đã sáp nhập với CBOT thành **CME Group**, cùng với *New York Mercantile Exchange*.

Trong lĩnh vực quyền chọn, *Chicago Board Options Exchange (CBOE)* bắt đầu giao dịch vào năm 1973 và hiện niêm yết hơn 2.500 mã cổ phiếu cùng nhiều chỉ số, đồng thời mở rộng sang quyền chọn trên tiền tệ và hợp đồng tương lai.

Giao dịch điện tử đang thay thế hệ thống *open outcry* truyền thống, giúp khớp lệnh tự động, nhanh chóng và hiệu quả hơn. Nhờ đó, các hình thức giao dịch tự động như *algorithmic trading*, *black-box trading* và *high-frequency trading (HFT)* ngày càng phát triển, đóng vai trò thiết yếu trong thị trường hiện đại.

2.1.2 Hợp đồng kỳ hạn (Forward Contracts)

Hợp đồng kỳ hạn là công cụ phái sinh đơn giản, đại diện cho thỏa thuận giữa hai bên về việc mua hoặc bán một tài sản vào một thời điểm xác định trong tương lai với mức giá đã thỏa thuận, khác với hợp đồng giao ngay vốn thực hiện ngay lập tức.

Chúng thường được giao dịch trên thị trường phi tập trung (OTC), giữa các tổ chức tài chính hoặc với khách hàng của họ. Một bên giữ vị thế **mua (long)**, bên còn lại giữ vị thế **bán (short)**.

Một ví dụ phổ biến là hợp đồng kỳ hạn ngoại hối: ngày 24/5/2010, một công ty Mỹ biết sẽ cần 1 triệu GBP sau 6 tháng có thể ký hợp đồng mua GBP theo tỷ giá kỳ hạn 1.4422 USD/GBP để phòng ngừa rủi ro tỷ giá.

Lợi nhuận (Payoff) từ hợp đồng kỳ hạn:

Với vị thế mua, lợi nhuận tại thời điểm đáo hạn được tính theo công thức:

$$\text{Lợi nhuận}_{\text{mua}} = S_T - K,$$

Với vị thế bán, lợi nhuận là:

$$\text{Lợi nhuận}_{\text{bán}} = K - S_T,$$

trong đó K là giá giao hàng đã thỏa thuận, và S_T là giá giao ngay tại thời điểm đáo hạn hợp đồng.

Lợi nhuận có thể là dương hoặc âm. Vì việc tham gia hợp đồng kỳ hạn thường không cần trả trước chi phí, nên lợi nhuận cũng chính là tổng lãi hoặc lỗ từ hợp đồng.

Giá kỳ hạn và giá giao ngay:
Giá kỳ hạn có mối liên hệ chặt chẽ với giá giao ngay. Trong thị trường không tồn tại cơ hội kinh doanh chênh lệch (arbitrage), nếu một tài sản không trả cổ tức, giá kỳ hạn F cho thời gian T năm được tính bằng:

$$F = S(1 + r)^T,$$

trong đó S là giá giao ngay, và r là lãi suất phi rủi ro. Nếu giá kỳ hạn cao hơn hoặc thấp hơn giá này sẽ tạo ra cơ hội arbitrage để kiếm lời không rủi ro.

2.1.3 Hợp đồng tương lai (Futures Contracts)

Hợp đồng tương lai cũng là thỏa thuận mua hoặc bán tài sản tại một thời điểm trong tương lai với mức giá cố định, nhưng khác với hợp đồng kỳ hạn ở chỗ chúng được giao dịch trên sàn tập trung và được chuẩn hóa về quy mô, ngày đáo hạn, tài sản cơ sở,... Sàn giao dịch và trung tâm thanh toán bù trừ đảm bảo việc thực hiện hợp đồng, giúp tăng thanh khoản và giảm rủi ro đối tác.

Hai sàn lớn nhất là **CBOT** và **CME**, nay thuộc **CME Group**, giao dịch nhiều loại tài sản như:

- **Hàng hóa**: vàng, đường, gỗ, thiếc, thịt bò, thịt lợn,...
- **Tài chính**: chỉ số chứng khoán, tiền tệ, trái phiếu,...

Ví dụ, ngày 1/9, giá hợp đồng tương lai vàng tháng 12 là $1.080/oz — mức giá phản ánh cung cầu thị trường tại thời điểm đó.

2.1.4 Options (Hợp đồng quyền chọn)

Hợp đồng quyền chọn là công cụ tài chính được giao dịch trên cả sàn giao dịch tập trung và thị trường phi tập trung (OTC). Có hai loại quyền chọn chính:

- **Call option (quyền chọn mua)**: Cho phép người nắm giữ được quyền (nhưng không bắt buộc) mua một tài sản cơ sở tại một mức giá xác định (giá thực hiện) vào hoặc trước một thời điểm trong tương lai (ngày đáo hạn).
- **Put option (quyền chọn bán)**: Cho phép người nắm giữ được quyền bán tài sản cơ sở tại một mức giá xác định vào hoặc trước ngày đáo hạn.

Có hai kiểu thực hiện:

- **American option**: Có thể được thực hiện bất kỳ lúc nào trước ngày đáo hạn.
- **European option**: Chỉ được thực hiện vào đúng ngày đáo hạn.

Khác với hợp đồng kỳ hạn hay hợp đồng tương lai (bắt buộc mua/bán), quyền chọn chỉ trao quyền chứ không bắt buộc thực hiện. Tuy nhiên, người mua quyền chọn phải trả một khoản phí (premium), trong khi hợp đồng tương lai/kỳ hạn thì không.

Sàn giao dịch quyền chọn lớn nhất thế giới là Chicago Board Options Exchange (CBOE). Ở đây, mỗi hợp đồng quyền chọn cổ phiếu thông thường tương ứng với 100 cổ phiếu cơ sở. Ví dụ, ngày 15/06/2010, một quyền chọn mua cổ phiếu Google (GOOG) đáo hạn tháng 12 với giá thực hiện $520 có giá chào bán là $32.00. Điều này đồng nghĩa với việc nhà đầu tư phải trả $3,200 để có quyền mua 100 cổ phiếu Google ở mức giá $520.

Ví dụ 1 (Long Call): Nếu giá cổ phiếu GOOG tăng lên $600 vào ngày đáo hạn, nhà đầu tư có thể mua 100 cổ phiếu với giá $520 và bán ra ngay với giá $600, thu lợi nhuận:

$$\$8,000 - \$3,200 = \$4,800.$$

Ngược lại, nếu giá cổ phiếu không vượt qua $520, quyền chọn không có giá trị và nhà đầu tư mất toàn bộ $3,200.

Ví dụ 2 (Short Put): Nếu nhà đầu tư bán một quyền chọn bán GOOG tháng 9 với giá thực hiện $480 (giá bid $22.20), họ thu được ngay $2,220. Nếu giá cổ phiếu ở thời điểm đáo hạn lớn hơn $480, quyền chọn không được thực hiện và nhà đầu tư giữ toàn bộ khoản tiền này. Nhưng nếu giá giảm xuống $420, nhà đầu tư sẽ phải mua 100 cổ phiếu ở mức giá $480, lỗ:

$$(\$480 - \$420) \times 100 = \$6,000.$$

Lợi nhuận ròng sau khi trừ khoản thu ban đầu là: $-\$3,780$.

Đặc điểm giá quyền chọn:

- Giá quyền chọn mua (*call*) giảm khi giá thực hiện tăng.
- Giá quyền chọn bán (*put*) tăng khi giá thực hiện tăng.
- Giá trị của cả hai loại quyền chọn đều tăng theo thời gian đáo hạn.

Các bảng giá quyền chọn điển hình (ví dụ như Bảng 1.2 và 1.3) thể hiện rõ mối quan hệ giữa giá thực hiện, thời gian đáo hạn và giá quyền chọn. Các yếu tố này sẽ được phân tích kỹ hơn trong các chương sau.

Các đối tượng tham gia thị trường quyền chọn gồm:

1. Người mua quyền chọn mua (long call).
2. Người bán quyền chọn mua (short call / writer).
3. Người mua quyền chọn bán (long put).
4. Người bán quyền chọn bán (short put / writer).

Trong đó, người mua có vị thế *long*, người bán có vị thế *short*. Việc bán quyền chọn thường được gọi là *writing an option*.

2.1.5 Types of Traders (Các loại nhà giao dịch)

Thị trường phái sinh đã phát triển vượt bậc nhờ vào việc thu hút được nhiều loại nhà giao dịch khác nhau và đạt mức thanh khoản cao. Nhờ đó, khi một nhà đầu tư muốn tham gia một phía của hợp đồng, thường sẽ dễ dàng tìm được người chấp nhận phía còn lại.

Có ba nhóm nhà giao dịch chính trên thị trường phái sinh:

- **Hedgers (Người phòng ngừa rủi ro)**: Sử dụng các công cụ phái sinh để giảm thiểu rủi ro từ biến động giá cả trong tương lai. Ví dụ: Doanh nghiệp nhập khẩu sử dụng hợp đồng kỳ hạn để khóa tỷ giá khi mua hàng hóa quốc tế.
- **Speculators (Nhà đầu cơ)**: Dùng phái sinh để đặt cược vào xu hướng của thị trường, nhằm thu lợi nhuận từ các biến động giá.
- **Arbitrageurs (Nhà kinh doanh chênh lệch giá)**: Tận dụng sự chênh lệch giá giữa hai hay nhiều thị trường/phái sinh để tạo ra lợi nhuận không rủi ro bằng cách thực hiện các giao dịch bù trừ (offsetting positions).

2.1.6 Phương trình Black-Scholes-Merton

Phương trình vi phân Black-Scholes-Merton (BSM) được phát triển từ việc xây dựng một danh mục đầu tư bao gồm cổ phiếu và quyền chọn, được điều chỉnh liên

tục để loại bỏ rủi ro trong khoảng thời gian vô cùng nhỏ. Phương trình này có dạng:

$$\frac{\partial f}{\partial t} + rS\frac{\partial f}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 f}{\partial S^2} = rf \tag{2.1}$$

Trong đó:

- f: giá của công cụ phái sinh (derivative),
- S: giá cổ phiếu cơ sở,
- t: thời gian,
- r: lãi suất phi rủi ro liên tục,
- σ: độ biến động (volatility) của giá cổ phiếu.

Điều kiện biên (boundary conditions) để giải phương trình phụ thuộc vào loại quyền chọn. Ví dụ:

- Đối với quyền chọn mua châu Âu (European Call): $f = \max(S - K, 0)$ tại $t = T$.
- Đối với quyền chọn bán châu Âu (European Put): $f = \max(K - S, 0)$ tại $t = T$.

Công thức định giá quyền chọn Black-Scholes-Merton

Khi giải phương trình vi phân (2.1) với điều kiện biên phù hợp, ta thu được các công thức định giá quyền chọn châu Âu nổi tiếng:

$$c = S_0 N(d_1) - Ke^{-rT} N(d_2), \tag{2.2}$$

$$p = Ke^{-rT} N(-d_2) - S_0 N(-d_1), \tag{2.3}$$

Trong đó:

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}},$$
$$d_2 = d_1 - \sigma\sqrt{T},$$

Các ký hiệu:

- S_0: giá cổ phiếu hiện tại,
- K: giá thực hiện (strike price),
- r: lãi suất phi rủi ro,

- σ: độ biến động,
- T: thời gian đáo hạn,
- $N(x)$: hàm phân phối tích lũy chuẩn (CDF) của phân phối chuẩn hoá $\mathcal{N}(0,1)$.

Giải thích xác suất và ý nghĩa kinh tế

- $N(d_2)$ đại diện cho xác suất thực hiện quyền chọn trong thế giới trung lập rủi ro (risk-neutral world).
- $S_0N(d_1)$ là giá trị kỳ vọng chiết khấu của payoff khi quyền chọn được thực hiện.
- $Ke^{-rT}N(d_2)$ là giá trị kỳ vọng chiết khấu của khoản thanh toán K.

Đặc tính của công thức Black-Scholes-Merton

Công thức BSM có các thuộc tính hợp lý với các giới hạn cực đoan:

- Khi $S_0 \to \infty$, giá call $\to S_0 - Ke^{-rT}$, giống như hợp đồng kỳ hạn.
- Khi $S_0 \to 0$, giá call $\to 0$, giá put $\to Ke^{-rT}$.
- Khi $\sigma \to 0$, giá call $\to \max(S_0 - Ke^{-rT}, 0)$, giá put $\to \max(Ke^{-rT} - S_0, 0)$.

Thực hành định giá và giả định

- Khi áp dụng trong thực tế, lãi suất r thường là lãi suất phi rủi ro theo kỳ hạn T từ đường cong lợi suất (zero-coupon).
- Mô hình giả định thị trường không có cơ hội arbitrage, giá cổ phiếu tuân theo chuyển động Brown hình học và không trả cổ tức.
- Giá quyền chọn mua kiểu Mỹ (American Call) trên cổ phiếu không cổ tức bằng với quyền chọn kiểu châu Âu.

Phương pháp định giá thay thế: Định giá trung lập rủi ro

Giá quyền chọn cũng có thể được xác định bằng kỳ vọng trung lập rủi ro:

$$c = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\max(S_T - K, 0)] \tag{2.4}$$

Với $\mathbb{Q}$ là xác suất trong thế giới trung lập rủi ro. Cách tiếp cận này dẫn đến cùng kết quả như phương trình 2.2.

2.2 Lý thuyết Trò chơi và tiềm năng ứng dụng trong Bài toán Market Making

Kỷ nguyên của Lý thuyết Trò chơi hiện đại bắt đầu từ khi *John von Neumann* giới thiệu những khái niệm nền tảng về cân bằng trong trò chơi hai người tổng bằng không, thông qua các công trình nghiên cứu và đặc biệt là tác phẩm kinh điển *"Theory of Games and Economic Behavior"* (1944), đồng tác giả với nhà kinh tế học *Oskar Morgenstern*. Kể từ đó, Lý thuyết Trò chơi đã phát triển mạnh mẽ và trở thành một công cụ quan trọng trong nhiều lĩnh vực như kinh tế học, khoa học máy tính, khoa học chính trị, sinh học,... Các khái niệm cốt lõi như trò chơi tĩnh và trò chơi động, trò chơi hợp tác và phi hợp tác, trò chơi với thông tin đầy đủ và không đầy đủ, cũng như khái niệm cân bằng Nash, đã trở nên phổ biến và quen thuộc.

Phần này sẽ trình bày một số khái niệm cơ bản và kết quả chính yếu trong lý thuyết trò chơi tĩnh với thông tin đầy đủ. Các nội dung lý thuyết của phần này được chủ yếu tham khảo từ [13].

2.2.1 Trò chơi Dạng chuẩn tắc và Cân bằng Nash

Phần này trình bày các khái niệm cơ bản liên quan đến Trò chơi Dạng chuẩn tắc và khái niệm cân bằng Nash. Cân bằng Nash, được đặt theo tên nhà toán học người Mỹ *John Forbes Nash Jr.* (1928–2015) — người được trao giải Nobel Kinh tế năm 1994 — là một chuẩn mực thể hiện sự nhất quán lẫn nhau trong chiến lược của các người chơi. Khái niệm này có khả năng áp dụng rộng rãi hơn so với tiêu chuẩn do *von Neumann* và *Morgenstern* đề xuất trước đó, giúp mở rộng phạm vi phân tích của Lý thuyết Trò chơi trong nhiều loại bài toán khác nhau.

Dạng chuẩn tắc của trò chơi

Định nghĩa 2.2.1. [13] Dạng chuẩn tắc của một trò chơi gồm 3 thành phần:
- Tập hữu hạn các người chơi (players) $\phi = \{1, 2, ..., I\}$,
- Tập chiến lược thuần túy S_i cho mỗi người chơi i,
- Hàm lợi nhuận (phần thưởng) $u_i(s)$ cho mỗi cấu hình chiến lược $s = (s_1, s_2, ..., s_I)$.

Chú ý: Ta dùng thuật ngữ "đối thủ của người chơi i" để chỉ tất cả những người chơi khác người chơi i, ký hiệu những người này bởi $-i$.

Định nghĩa 2.2.2. [13] Một chiến lược hỗn hợp σ_i cho người chơi i là một phân phối xác suất trên tập chiến lược thuần túy S_i, thỏa mãn các phân phối xác suất này giữa các người chơi là độc lập toàn thể.
- Ký hiệu không gian tất cả các chiến lược hỗn hợp cho người chơi i bởi $\sum_i$, khi đó không gian các cấu hình hỗn hợp ký hiệu bởi $\sum = \times \sum_i$, với $\sigma = (\sigma_1, \sigma_2, ..., \sigma_I)$.
- Tựa của một chiến lược hỗn hợp σ_i là tập các chiến lược thuần túy của người chơi i được gắn xác suất dương bởi σ_i.

- Lợi nhuận của người chơi i đối với cấu hình σ, ký hiệu $u_i(\sigma)$, được tính bởi kỳ vọng của lợi nhuận đối với cấu hình thuần túy tương ứng:

$$u_i(\sigma) = \sum_{s \in S} \left(\prod_j \sigma_j(s_j) \right) u_i(s)$$

Ví dụ 1: Song đề Tù nhân (The Prison Dilemma)

	Cooperate	Defect
Cooperate	1, 1	-1, 2
Defect	2, -1	0, 0

Bảng 2.1: Ví dụ 1: Song đề Tù nhân

Một trong những ví dụ nổi tiếng nhất về Lý thuyết Trò chơi chính là Song đề Tù nhân (The Prison Dilemma). Câu chuyện đằng sau ví dụ này là 2 người đàn ông bị viên cảnh sát tạm giam do nghi ngờ phạm tội. Tuy nhiên do thiếu bằng chứng kết tội, viên cảnh sát đưa hai nghi phạm vào hai phòng tạm giam tách biệt, và thuyết phục hai người làm chứng chống lại người kia. Nếu cả 2 đều làm chứng chống lại người kia (Defect), họ sẽ đều bị phạt tù 5 năm. Còn nếu 1 trong 2 người (gọi là X) không làm chứng (Cooperate), còn người kia (Y) làm chứng (Defect), thì Y sẽ được thả tự do, còn X sẽ bị phạt tù nặng (10 năm). Trong trường hợp cả hai đều không làm chứng (Cooperate), họ sẽ chỉ bị phạt tù 2 năm cho 1 tội nhẹ hơn. Ví dụ dạng bảng trên chỉ là 1 phiên bản khác của Song đề Tù nhân dưới dạng thưởng/phạt.

Tiếp theo chúng ta giới thiệu một khái niệm quan trọng trong Lý thuyết Trò chơi: Cân bằng Nash.

Cân bằng Nash và ví dụ

Định nghĩa 2.2.3. [13] - Một cấu hình chiến lược hỗn hợp σ^* được gọi là 1 cân bằng Nash nếu với mọi người chơi i,

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(s_i, \sigma_{-i}^*), \forall s_i \in S_i,$$

- Một cấu hình chiến lược cân bằng Nash thuần túy đơn giản là một cấu hình chiến lược cân bằng Nash hỗn hợp suy biến.
- Một cấu hình chiến lược thuần túy s^* được gọi là cân bằng Nash chặt nếu với mọi i và với $s_i \neq s_i^*$,

$$u_i(s_i^*, s_{-i}^*) > u_i(s_i, s_{-i}^*)$$

Nhận xét:
Cân bằng Nash là khái niệm trung tâm trong lý thuyết trò chơi, mô tả một trạng thái ổn định của trò chơi trong đó không người chơi nào có động lực đơn phương thay đổi chiến lược của mình. Cụ thể, một cấu hình chiến lược hỗn hợp $\sigma^* = (\sigma_1^*, \ldots, \sigma_n^*)$ là một *cân bằng Nash* nếu với mọi người chơi i và mọi chiến lược $s_i \in S_i$, ta có:

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(s_i, \sigma_{-i}^*)$$

Nói cách khác, nếu mọi người chơi khác giữ nguyên chiến lược của họ, thì người chơi i không thể cải thiện lợi ích kỳ vọng của mình bằng cách chuyển sang một chiến lược khác. Điều này phản ánh nguyên tắc cơ bản: *mỗi người chơi đang hành động một cách tối ưu có tính đến chiến lược của những người chơi còn lại*.

Ví dụ 2: Trò chơi bầu cử (Tồn tại cân bằng Nash thuần túy)
Có 3 người chơi 1,2 và 3 bầu cho 3 ứng cử viên A, B, C. Mỗi người chơi bầu cho đúng một ứng cử viên, phiếu trắng không được chấp nhận. Ứng cử viên chiếm nhiều phiếu nhất giành chiến thắng, ngược lại, nếu không có ứng cử viên chiếm đa số, ứng cử viên A sẽ được lựa chọn. Hàm lợi nhuận của các người chơi là:

$$u_1(A) = u_2(B) = u_3(C) = 2,$$

$$u_1(B) = u_2(C) = u_3(A) = 1,$$

$$u_1(C) = u_2(A) = u_3(B) = 0.$$

Trò chơi này có nhiều điểm cân bằn. Chẳng hạn, nếu người chơi 1 và 3 cùng bỏ phiếu cho phương án A, thì lá phiếu của người chơi 2 sẽ không làm thay đổi kết quả, và người chơi 3 không có sự ưu tiên rõ ràng về cách bỏ phiếu. Do đó, các cấu hình (A, A, A) và (A, B, A) đều là các cân bằng Nash mà kết quả cuối cùng là A.

Mặt khác, cấu hình (A, A, B) không phải là một cân bằng Nash, bởi vì nếu người chơi 3 bỏ phiếu cho B, thì người chơi 2 sẽ muốn chuyển sang bỏ phiếu cho B để kết quả cũng là B.

Ví dụ 3: Trò chơi chọn mặt đồng xu (Không tồn tại cân bằng Nash thuần túy)
Hai người chơi 1 và 2 cùng lúc chọn 1 mặt đồng xu $H(head)$ hoặc $T(tail)$. Nếu 2 mặt trùng nhau, người chơi 1 nhận được 1$ còn người chơi 2 mất 1$ và ngược lại. Dễ thấy nếu kết quả dự đoán dẫn tới 2 mặt trùng, sẽ dẫn tới người chơi 2 thay đổi sang chọn mặt còn lại và ngược lại khi kết cục dự đoán là 2 mặt không trùng, sẽ thúc đẩy người chơi 1 thay đổi sang mặt còn lại. Tình huống "ổn định" duy nhất

là 2 người chơi chọn ngẫu nhiên 1 trong 2 mặt. Thật vậy nếu người chơi 1 chọn ngẫu nhiên $\frac{1}{2} - \frac{1}{2}$ giữa H và T, lợi nhuận kỳ vọng của người chơi 2 khi chọn H là $0.5 \times (-1) + 0.5 \times 1 = 0$ và kỳ vọng khi chọn T là $0.5 \times 1 + 0.5 \times (-1) = 0$. Khi đó người chơi 2 sẽ phân vân giữa việc chọn H hay T và sẽ sẵn lòng chọn ngẫu nhiêu giữa 2 lựa chọn. Như vậy, trong trò chơi Chọn mặt đồng xu này, không tồn tại cân bằng Nash thuần túy mà chỉ tồn tại cân bằng Nash hỗn hợp.

Các định lý về sự tồn tại của cân bằng Nash

Phần tiếp theo, ta tìm hiểu các Định lý cơ bản về sự tồn tại của cân bằng Nash (thuần túy và hỗn hợp) trong các dạng trò chơi cơ bản (hữu hạn và vô hạn).

Định lý 2.2.1. (Định lý Kakatuni) [14] Giả sử S là một tập con khác rỗng, lồi, compact của một không gian Euclide R^n, và $\phi : S \to 2^S$ là ánh xạ thỏa mãn tính chất:
- ϕ có đồ thị đóng.
- $\phi(x)$ là tập lồi khác rỗng với mọi $x \in S$.
Khi đó ϕ tồn tại điểm bất động, nghĩa là tồn tại $x \in S$ sao cho $x \in \phi(x)$.

Định lý 2.2.2. (Nash) [15] Mọi trò chơi dạng chuẩn tắc hữu hạn đều có cấu hình chiến lược cân bằng Nash hỗn hợp.

Định lý 2.2.3. (1952, Debreu, Glicksberg, Fan) [16] Với trò chơi dạng chuẩn tắc trong đó không gian chiến lược S_i là tập con lồi, compact và khác rỗng của 1 không gian Euclide, nếu hàm lợi nhuận u_i liên tục theo s và là hàm tựa lõm theo s_i, khi đó tồn tại cấu hình chiến lược cân bằng Nash thuần túy.

Định lý 2.2.4. (Glicksberg 1952) [16] Với trò chơi dạng chuẩn tắc trong đó không gian chiến lược S_i là tập con compact khác rỗng của một không gian metric. Khi đó nếu các hàm u_i liên tục trên S thì tồn tại cấu hình chiến lược cân bằng Nash hỗn hợp.

Tiềm năng ứng dụng Lý thuyết Trò chơi trong bài toán Market Making

Lý thuyết Trò chơi cung cấp một khuôn khổ toán học mạnh mẽ để phân tích các tình huống có sự tương tác chiến lược giữa nhiều tác nhân. Trong bối cảnh thị trường tài chính, đặc biệt là trong hoạt động *Market Making*, các tác nhân (Market Makers, Arbitrageurs, Traders, ...) không hoạt động một cách độc lập, mà liên tục phản ứng với hành vi của nhau để tối ưu hóa lợi nhuận hoặc giảm thiểu rủi ro. Điều này làm cho bài toán Market Making trở thành một ví dụ điển hình của trò chơi nhiều

người chơi với lợi ích xung đột hoặc phụ thuộc lẫn nhau.

Một Market Maker (MM) không chỉ đưa ra quyết định đặt giá mua (bid) và bán (ask) dựa trên trạng thái thị trường tại một thời điểm, mà còn cần **dự đoán và phản ứng với hành vi của các MM khác**. Do đó, việc sử dụng Lý thuyết Trò chơi, đặc biệt là các khái niệm như *cân bằng Nash*, *trò chơi lặp lại*, hoặc *trò chơi thông tin không hoàn chỉnh*, có thể giúp mô hình hóa hành vi chiến lược của các Market Maker trong môi trường cạnh tranh cao.

Ngoài ra, trong các sàn giao dịch điện tử hiện đại, số lượng lớn các Market Maker sử dụng thuật toán để ra quyết định, dẫn đến môi trường có tính **lặp lại cao** và **tốc độ phản ứng rất nhanh**. Trong bối cảnh này, việc sử dụng Lý thuyết Trò chơi không chỉ giúp mô hình hóa trò chơi chiến lược giữa các tác nhân mà còn hỗ trợ *thiết kế chiến lược tối ưu* dựa trên phản hồi liên tục từ thị trường. Điều này đặc biệt phù hợp với việc tích hợp các mô hình học tăng cường (Reinforcement Learning) để học chiến lược tối ưu qua tương tác.

Tóm lại, Lý thuyết Trò chơi mở ra hướng tiếp cận có tính nền tảng và hệ thống trong việc hiểu và xây dựng chiến lược Market Making, giúp các tác nhân không chỉ phản ứng với thị trường mà còn **dự đoán và điều chỉnh theo chiến lược của đối thủ**, từ đó nâng cao hiệu quả giao dịch và khả năng kiểm soát rủi ro.

2.3 Thuật toán K-Means trong phân cụm dữ liệu

Phân cụm (clustering) là một trong những kỹ thuật cơ bản và quan trọng trong phân tích dữ liệu. Mục tiêu của phân cụm là chia một tập dữ liệu thành các nhóm (clusters) sao cho các điểm dữ liệu trong cùng một nhóm thì tương đồng với nhau, còn giữa các nhóm thì có sự khác biệt rõ ràng. Phân cụm được ứng dụng trong nhiều bài toán thực tế như rút gọn dữ liệu, truy xuất thông tin hiệu quả, nhận diện chủ đề chính trong dữ liệu, hoặc là bước tiền xử lý trong học máy.

Một trong những thuật toán phân cụm phổ biến và lâu đời nhất là **K-Means**, được đề xuất lần đầu bởi Lloyd (1957) và độc lập bởi Steinhaus (1956). Tính đến nay, K-Means vẫn là thuật toán được sử dụng rộng rãi nhất trong các ứng dụng phân cụm nhờ sự đơn giản và hiệu quả tính toán. Tuy nhiên, thuật toán này cũng tồn tại một số hạn chế nhất định liên quan đến hội tụ tại cực trị cục bộ và độ nhạy với lựa chọn khởi tạo ban đầu.

Trong bối cảnh thị trường tài chính hiện đại, đặc biệt là trên các sàn giao dịch điện tử, hoạt động giao dịch ngày càng được tự động hóa thông qua các bot với nhiều chiến lược khác nhau. Việc phân tích và nhận diện hành vi các bot này là yếu tố then chốt để hiểu được cấu trúc thị trường và xây dựng chiến lược phản ứng phù hợp. Trong đồ án này, thuật toán K-Means được sử dụng như một công cụ cốt lõi để phân cụm các lệnh đặt theo đặc trưng hành vi, từ đó suy luận ra nhóm bot có hành vi tương đồng. Đây là bước khởi đầu quan trọng giúp mô hình hóa chiến lược của các đối thủ, tạo tiền đề cho việc phân tích cân bằng chiến lược và huấn luyện tác nhân Market Making hiệu quả hơn.

2.3.1 Bài toán tối ưu tổng sai số bình phương (SSE)

K-Means được thiết kế để giải bài toán tối ưu hàm mục tiêu là **tổng sai số bình phương** (Sum of Squared Errors - SSE). Cho một tập dữ liệu gồm n điểm trong không gian Euclid $\mathbb{R}^d$, bài toán yêu cầu tìm k tâm cụm $C = \{c_1, ..., c_k\} \subset \mathbb{R}^d$ sao cho tổng bình phương khoảng cách từ mỗi điểm đến tâm gần nhất được tối thiểu hóa:

$$\text{cost}(P, C) := \sum_{p \in P} \min_{c \in C} \|p - c\|^2 \tag{2.5}$$

Trong đó:

- $P \subset \mathbb{R}^d$ là tập dữ liệu đầu vào,
- C là tập các tâm cụm cần tìm,
- $\| \cdot \|^2$ là khoảng cách Euclid bình phương.

Việc lựa chọn số cụm k là một giả định đầu vào, có thể xác định bằng kiến thức chuyên ngành hoặc thông qua các tiêu chí đánh giá như phương pháp khuỷu tay (Elbow method), hệ số Silhouette, v.v.

2.3.2 Mô tả thuật toán K-Means

K-Means là một thuật toán lặp cải tiến cục bộ (local improvement heuristic) với hai bước chính được lặp lại cho đến khi hội tụ.

Thuật toán K-Means

- **Input:** Tập dữ liệu $P \subset \mathbb{R}^d$, số cụm k.
- **Output:** Tập tâm cụm $C = \{c_1, ..., c_k\}$.

1. Khởi tạo ngẫu nhiên k tâm cụm $\{c_1, ..., c_k\} \subset \mathbb{R}^d$.
2. Lặp lại cho đến khi các tâm cụm không thay đổi:
   - Gán mỗi điểm $p \in P$ vào cụm gần nhất (tính theo khoảng cách Euclid)
   - Cập nhật mỗi tâm cụm c_i là trung bình cộng của các điểm trong cụm i:

$$c_i = \frac{1}{|P_i|} \sum_{p \in P_i} p$$

Quá trình lặp này luôn làm giảm giá trị hàm mục tiêu và đảm bảo hội tụ tại một điểm dừng (thường là cực trị cục bộ).

2.3.3 Đặc điểm và đánh giá

Ưu điểm:

- Đơn giản, dễ cài đặt.
- Tính toán hiệu quả với dữ liệu lớn.
- Hiệu quả với các cụm hình cầu và mật độ đồng đều.
- Có thể mở rộng thành các biến thể nhanh hơn và chính xác hơn.

Nhược điểm:

- Phụ thuộc mạnh vào khởi tạo ban đầu.
- Dễ hội tụ tại cực trị cục bộ.
- Không xử lý tốt các cụm có hình dạng phức tạp hoặc mật độ không đều.
- Cần biết trước số lượng cụm k.

2.3.4 Các cải tiến và mở rộng

Nhiều phương pháp đã được đề xuất để cải thiện thuật toán K-Means, nổi bật như:

- **K-Means++**: khởi tạo thông minh hơn để tăng khả năng hội tụ toàn cục.
- **Mini-Batch K-Means**: cải thiện tốc độ với dữ liệu lớn bằng cách sử dụng minibatch.
- **Soft K-Means**: cho phép phân cụm mờ, mỗi điểm thuộc về nhiều cụm với xác suất.
- **Bregman K-Means**: mở rộng cho các hàm phân kỳ Bregman thay vì khoảng cách Euclid.

2.4 Thuật toán Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG) là một thuật toán học tăng cường sâu (Deep Reinforcement Learning) được thiết kế cho các bài toán có không gian hành động liên tục, vốn phổ biến trong các ứng dụng thực tế như robot điều khiển, tài chính và đặc biệt là trong các hệ thống giao dịch tự động như Market Making.

DDPG được đề xuất lần đầu bởi Lillicrap et al. (2016) [17], kết hợp ưu điểm của hai phương pháp nổi bật:

- **Deterministic Policy Gradient (DPG)**: sử dụng chính sách quyết định (deterministic policy), tức đầu ra của policy là một hành động cụ thể thay vì phân phối hành động.
- **Deep Q-Network (DQN)**: sử dụng replay buffer và target network để tăng tính ổn định trong quá trình huấn luyện mạng neuron.

Trong bài toán Market Making, không gian hành động (ví dụ: spread bid/ask, volume) là liên tục và nhiều chiều, vì vậy DDPG là lựa chọn phù hợp hơn so với các thuật toán rời rạc như DQN.

2.4.1 Cơ sở lý thuyết của thuật toán DDPG

Ta xét một thiết lập chuẩn của học tăng cường, trong đó một tác nhân (agent) tương tác với môi trường $\mathcal{E}$ theo các bước thời gian rời rạc. Ở mỗi thời điểm t, tác nhân sẽ nhận được một quan sát x_t, lựa chọn một hành động a_t và nhận về một phần thưởng vô hướng r_t.

Trong trường hợp tổng quát, môi trường có thể không được quan sát đầy đủ, do đó trạng thái tại thời điểm t có thể cần phải dựa vào toàn bộ lịch sử quan sát và hành động trước đó, được biểu diễn là

$$s_t = (x_1, a_1, \ldots, a_{t-1}, x_t)$$

Tuy nhiên, trong bài toán này, giả định rằng môi trường được quan sát đầy đủ, tức là trạng thái s_t chỉ đơn giản là quan sát hiện tại x_t.

Chính sách (policy) của tác nhân được định nghĩa như một hàm ánh xạ từ trạng thái tới phân phối xác suất trên không gian hành động:

$$\pi : \mathcal{S} \to \mathcal{P}(\mathcal{A})$$

Môi trường $\mathcal{E}$ có thể bao gồm sự ngẫu nhiên, và được mô hình hóa như một quá trình quyết định Markov (Markov Decision Process - MDP), với các thành phần gồm:

- Không gian trạng thái $\mathcal{S}$,
- Không gian hành động $\mathcal{A} = \mathbb{R}^N$ (với hành động là vectơ thực N chiều),
- Phân phối trạng thái khởi đầu $p(s_1)$,
- Động lực chuyển tiếp $p(s_{t+1} \mid s_t, a_t)$,
- Hàm phần thưởng $r(s_t, a_t)$.

Tổng phần thưởng (return) từ trạng thái s_t được tính theo công thức chiết khấu:

$$R_t = \sum_{i=t}^{T} \gamma^{i-t} r(s_i, a_i),$$

trong đó $\gamma \in [0, 1]$ là hệ số chiết khấu thể hiện mức độ ưu tiên cho phần thưởng ở tương lai gần hơn.

Mục tiêu của học tăng cường là tìm một chính sách tối ưu sao cho kỳ vọng tổng phần thưởng bắt đầu từ phân phối trạng thái ban đầu là lớn nhất:

$$J = \mathbb{E}_{r_i, s_i \sim \mathcal{E}, a_i \sim \pi}[R_1]$$

Phân phối tần suất chiết khấu của các trạng thái được thăm bởi chính sách π được ký hiệu là ρ_π.

Hàm giá trị hành động (action-value function) $Q_\pi(s, a)$ mô tả kỳ vọng tổng phần thưởng bắt đầu từ trạng thái s và thực hiện hành động a, rồi theo chính sách π tiếp theo:

$$Q_\pi(s_t, a_t) = \mathbb{E}_{r_i, s_i, a_i \sim \mathcal{E}, \pi}[R_t \mid s_t, a_t]$$

Phương trình Bellman thiết lập mối quan hệ đệ quy cho hàm Q:

$$Q_\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim \mathcal{E}} \left[r(s_t, a_t) + \gamma \, \mathbb{E}_{a_{t+1} \sim \pi} \left[Q_\pi(s_{t+1}, a_{t+1}) \right] \right]$$

Khi chính sách là hàm quyết định (deterministic), ký hiệu là $\mu : \mathcal{S} \to \mathcal{A}$, ta có thể viết lại:

$$Q_\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim \mathcal{E}} \left[r(s_t, a_t) + \gamma Q_\mu(s_{t+1}, \mu(s_{t+1})) \right].$$

Hàm Q này có thể được học theo phương pháp *off-policy*, tức là dùng dữ liệu thu thập được từ một chính sách khác.

Q-learning là một thuật toán off-policy phổ biến, trong đó chính sách được cập

nhật theo quy tắc tham lam:

$$\mu(s) = \arg\max_a Q(s, a).$$

Trong thực tế, các hàm Q được xấp xỉ bởi các mô hình tham số, ví dụ mạng nơ-ron với tham số θ_Q, và được học bằng cách tối thiểu hóa hàm mất mát:

$$L(\theta_Q) = \mathbb{E}_{s_t, a_t, r_t}\left[\left(Q(s_t, a_t \mid \theta_Q) - y_t\right)^2\right],$$

với mục tiêu:

$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) \mid \theta_Q)$$

Việc áp dụng Q-learning vào không gian hành động liên tục gặp nhiều khó khăn, do việc tìm hành động tối ưu tại mỗi bước là bài toán tối ưu phi tuyến phức tạp và tốn kém.

Để giải quyết vấn đề này, người ta sử dụng phương pháp Actor-Critic dựa trên thuật toán *Deterministic Policy Gradient* (DPG). Trong đó:

- **Actor** là một hàm tham số $\mu(s \mid \theta_\mu)$, ánh xạ trạng thái sang hành động xác định,
- **Critic** là hàm $Q(s, a)$ được học dựa trên phương trình Bellman.

Quá trình cập nhật Actor được thực hiện bằng cách tính đạo hàm của hàm kỳ vọng phần thưởng J theo tham số θ_μ, dựa trên chuỗi đạo hàm (chain rule):

$$\nabla_{\theta_\mu} J \approx \mathbb{E}_{s_t \sim \rho_\beta}\left[\nabla_a Q(s, a \mid \theta_Q)\big|_{a=\mu(s)} \cdot \nabla_{\theta_\mu} \mu(s \mid \theta_\mu)\right]$$

Phương pháp này giúp học một chính sách liên tục trong không gian hành động phức tạp một cách hiệu quả và ổn định.

2.4.2 Cấu trúc tổng thể

DDPG là một thuật toán off-policy actor-critic, với hai mạng riêng biệt:

- **Actor network** $\mu(s|\theta_\mu)$: ánh xạ trạng thái sang hành động tối ưu.
- **Critic network** $Q(s, a|\theta_Q)$: ước lượng giá trị kỳ vọng của hành động trong trạng thái s.

Để tăng độ ổn định khi huấn luyện, DDPG sử dụng hai kỹ thuật quan trọng:

- **Replay buffer**: ghi lại các chuyển tiếp (s, a, r, s'), cho phép huấn luyện bằng các minibatch được lấy mẫu ngẫu nhiên.

- **Target networks** μ', Q': bản sao trễ của actor và critic, được cập nhật chậm hơn để giảm dao động và tránh gradient exploding.

2.4.3 Thuật toán chi tiết

Algorithm 1: Thuật toán DDPG

Khởi tạo các mạng:

- Actor: $\mu(s|\theta_\mu)$.
- Critic: $Q(s, a|\theta_Q)$.
- Mạng mục tiêu: μ', Q' với $\theta_{\mu'} \leftarrow \theta_\mu$, $\theta_{Q'} \leftarrow \theta_Q$.
- Replay Buffer $\mathcal{R}$.

for *mỗi episode* **do**

Khởi tạo trạng thái ban đầu s_0.

for *từng bước thời gian* **do**

Chọn hành động: $a = \mu(s|\theta_\mu) + \text{noise}$.

Thực hiện hành động a, nhận reward r và trạng thái mới s'.

Lưu (s, a, r, s') vào $\mathcal{R}$.

Lấy minibatch N mẫu từ $\mathcal{R}$.

Tính giá trị mục tiêu:

$$y = r + \gamma Q'(s', \mu'(s'|\theta_{\mu'})|\theta_{Q'})$$

Cập nhật Critic bằng gradient của lỗi MSE:

$$L = \frac{1}{N} \sum (y - Q(s, a|\theta_Q))^2$$

Cập nhật Actor bằng policy gradient:

$$\nabla_{\theta_\mu} J \approx \frac{1}{N} \sum \nabla_a Q(s, a|\theta_Q)\big|_{a=\mu(s)} \nabla_{\theta_\mu} \mu(s|\theta_\mu)$$

Cập nhật target networks:

$$\begin{cases} \theta_{\mu'} \leftarrow \tau\theta_\mu + (1-\tau)\theta_{\mu'} \\ \theta_{Q'} \leftarrow \tau\theta_Q + (1-\tau)\theta_{Q'} \end{cases}$$

2.4.4 Các thành phần quan trọng

- **Actor Network**: nhận đầu vào là trạng thái s, xuất ra hành động liên tục a. Mạng này được huấn luyện bằng gradient của Critic theo công thức:

$$\nabla_{\theta_\mu} J = \mathbb{E}\left[\nabla_a Q(s, a|\theta_Q)\big|_{a=\mu(s)} \cdot \nabla_{\theta_\mu}\mu(s|\theta_\mu)\right]$$

- **Critic Network**: nhận đầu vào (s, a), xuất ra giá trị $Q(s, a)$, được huấn luyện bằng cách tối thiểu hóa hàm mất mát MSE giữa giá trị mục tiêu y và giá trị ước lượng:

$$y = r + \gamma Q'(s', \mu'(s'))$$

- **Replay Buffer**: lưu trữ một lượng lớn các mẫu chuyển tiếp (s, a, r, s') (ví dụ 1 triệu mẫu) để decorrelate dữ liệu. Lấy mẫu minibatch ngẫu nhiên cho mỗi bước huấn luyện.
- **Target Network**: là bản sao trễ của Actor và Critic, cập nhật theo hàm trung bình động với tham số nhỏ $\tau \ll 1$ (thường khoảng 0.001):

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$$

Kết luận

Tổng kết lại, chương này đã trình bày các kiến thức nền tảng quan trọng làm cơ sở cho toàn bộ quá trình nghiên cứu và triển khai mô hình trong đồ án. Từ vai trò của công cụ tài chính phái sinh và chiến lược Market Making trong giao dịch hiện đại, đến các lý thuyết cốt lõi như Lý thuyết Trò chơi, thuật toán K-Means và phương pháp học tăng cường DDPG, mỗi nội dung đều đóng vai trò thiết yếu trong việc xây dựng một hệ thống phân tích và ra quyết định chiến lược hiệu quả. Những kiến thức này sẽ tiếp tục được vận dụng và phát triển trong các chương sau, nơi các thuật toán được tích hợp vào mô hình thực nghiệm cụ thể nhằm giải quyết bài toán Market Making một cách có hệ thống và tối ưu.

CHƯƠNG 3. PHƯƠNG PHÁP LUẬN NGHIÊN CỨU

Trong chương này, tác giả trình bày toàn bộ quy trình kỹ thuật được thiết kế nhằm giải quyết bài toán Market Making bằng cách kết hợp học tăng cường sâu (Reinforcement Learning) với Lý thuyết Trò chơi. Cách tiếp cận của đồ án không chỉ dừng lại ở việc huấn luyện một tác nhân giao dịch thông minh, mà còn tập trung xây dựng một pipeline phân tích thị trường có khả năng thích ứng liên tục với môi trường giao dịch thực tế. Toàn bộ quy trình được triển khai trên dữ liệu thật từ sàn Binance, bao gồm các bước thu thập dữ liệu theo thời gian thực, phân tích hành vi thị trường, mô hình hóa chiến lược đối thủ và học hỏi thông qua tương tác chiến lược.

Phần đầu chương trình bày quy trình xử lý dữ liệu order book thô thành các snapshot có cấu trúc ổn định, làm đầu vào cho các bước phân tích tiếp theo. Sau đó, một hệ thống phân cụm được phát triển để phát hiện các nhóm hành vi bot khác nhau, từ đó nhận diện các chiến lược đặc thù mà từng nhóm đang theo đuổi. Dựa trên các chiến lược này, một khung trò chơi được hình thành để mô phỏng tương tác giữa các tác nhân, trong đó các thuật toán từ Lý thuyết Trò chơi được sử dụng để tìm điểm cân bằng chiến lược.

Trên cơ sở đó, tác giả thiết kế một môi trường mô phỏng Market Making có tích hợp các yếu tố chiến lược từ đối thủ, cho phép huấn luyện một tác nhân DDPG trong không gian hành động liên tục. Tác nhân này được học cách đưa ra quyết định đặt lệnh tối ưu bằng cách khai thác sự chênh lệch giữa trạng thái thị trường hiện tại và chiến lược cân bằng. Cuối chương này mô tả cách hệ thống được kiểm thử và đánh giá trên dữ liệu thực tế, sử dụng các chỉ số tài chính như PnL và hệ số Sharpe để đo lường hiệu quả tác nhân. Phần này đóng vai trò kiểm chứng thực nghiệm cho toàn bộ phương pháp được đề xuất.

3.1 Tổng quan

Giải pháp được đề xuất trong nghiên cứu này nhằm mục tiêu xây dựng một hệ thống Market Making tự động, trong đó một tác nhân học tăng cường (Reinforcement Learning Agent) có thể học cách đặt lệnh hiệu quả trong một môi trường cạnh tranh khốc liệt giữa các Market Maker trên sàn giao dịch Binance. Để làm được điều này, tác giả kết hợp phương pháp học tăng cường sâu (Deep Reinforcement Learning) với lý thuyết trò chơi (Game Theory), đồng thời xử lý dữ liệu thị trường thực thông qua phân tích order book.

Toàn bộ giải pháp bao gồm một chuỗi các bước được thực hiện lặp lại theo thời gian

thực (real-time pipeline), mô phỏng quá trình tác nhân tương tác với môi trường và học hỏi chiến lược tối ưu. Cụ thể, mỗi vòng lặp (episode hoặc step) trong hệ thống sẽ thực hiện các bước chính sau:

- Thu thập dữ liệu thị trường: Lấy một snapshot order book từ Binance thông qua WebSocket API theo thời gian thực, bao gồm các lệnh giới hạn ở nhiều mức giá (level 1 đến level N).
- Phân cụm các lệnh (Order Clustering): Áp dụng thuật toán phân cụm (ví dụ: KMeans) trên các đặc trưng của lệnh (thời gian sống, khoảng cách đến midprice, khối lượng, số lần cập nhật, v.v.) để nhóm các lệnh vào từng cụm hành vi – mỗi cụm được xem như đại diện cho một bot Market Maker khác nhau.
- Nhận diện chiến lược từng bot: Phân tích đặc điểm của từng cụm để xác định chiến lược của từng bot (ví dụ: narrow spread, aggressive buy, passive sell...) và trích xuất các tham số đặc trưng như mức spread trung bình, khối lượng tương đối, độ thiên lệch mua/bán,...
- Xây dựng ma trận payoff và tìm cân bằng Nash: Dựa trên các chiến lược nhận diện được, xây dựng ma trận payoff giữa các bot, sau đó sử dụng thuật toán phản hồi tốt nhất (Best Response Dynamics) để tìm điểm cân bằng Nash gần đúng. Qua đó, xác định phần thưởng tiềm năng mà một chiến lược cụ thể có thể nhận được trong môi trường hiện tại.
- Sinh quan sát (observation): Tổng hợp thông tin từ order book, chiến lược của các bot khác, và lịch sử phần thưởng để tạo ra một vector quan sát đầu vào cho agent.
- Tác nhân RL chọn hành động: Agent sử dụng mô hình DDPG (Deep Deterministic Policy Gradient) để sinh ra hành động bao gồm các thông số như mức spread, volume đặt lệnh và mức độ chấp nhận rủi ro.
- Tính phần thưởng (reward): Reward tại mỗi bước được xác định là payoff mà agent đáng lẽ sẽ nhận được nếu áp dụng chiến lược Nash ở bước trước vào tình huống thị trường hiện tại, dựa trên mô hình cạnh tranh với các bot khác.
- Huấn luyện mô hình RL: Agent cập nhật chính sách hành động của mình dựa trên trải nghiệm vừa thu được (observation, action, reward, next observation), lưu trữ vào bộ nhớ kinh nghiệm (replay buffer) và huấn luyện định kỳ.

Nhờ vào pipeline trên, agent có thể học liên tục trong môi trường gần thực tế, vừa phản ứng với hành vi của các Market Maker khác, vừa điều chỉnh chiến lược nhằm tối ưu hóa phần thưởng.

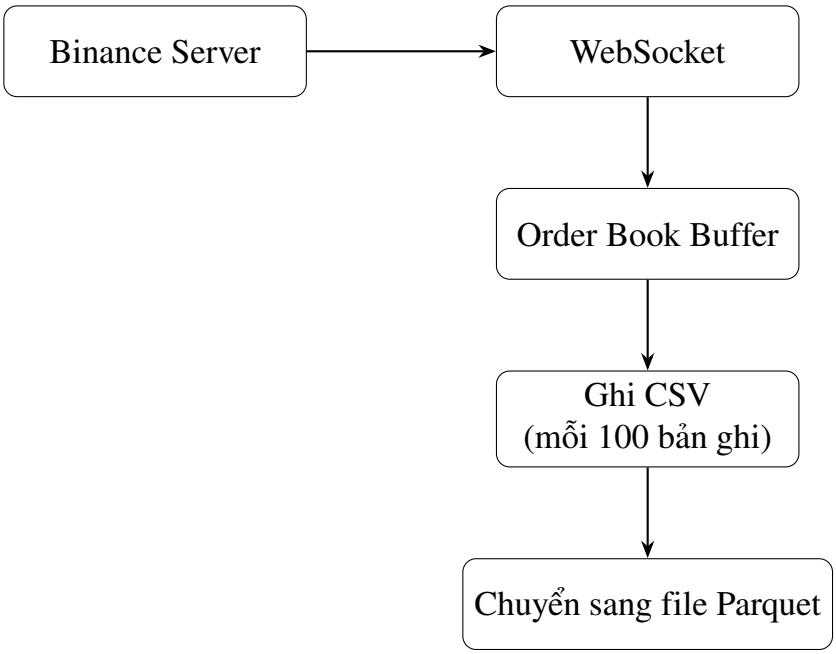
Dưới đây là sơ đồ mô tả tổng quan luồng hoạt động của hệ thống:

3.2 Thu thập dữ liệu thị trường

3.2.1 Mục tiêu

Mục tiêu của bước này là thu thập dữ liệu Order Book từ sàn Binance theo thời gian thực bằng WebSocket API, đồng thời đảm bảo độ liên tục của dữ liệu thông qua kết hợp với REST API. Dữ liệu sau đó được chuẩn hoá và lưu trữ hiệu quả dưới dạng file Parquet để phục vụ các bước xử lý chiến lược và huấn luyện Reinforcement Learning sau này.

3.2.2 Kiến trúc hệ thống thu thập dữ liệu



Hình 3.1: Quy trình thu thập và lưu trữ dữ liệu từ Binance

3.2.3 Thuyết minh các bước thực hiện

Bước 1: Lấy snapshot khởi tạo (REST API)

Do WebSocket chỉ cung cấp dữ liệu delta (chênh lệch), nên cần lấy trạng thái toàn bộ order book ban đầu bằng REST API trước khi bắt đầu nghe WebSocket để đảm bảo dữ liệu chính xác và đầy đủ.

Listing 3.1: Lấy snapshot từ REST API

```
function get_initial_order_book():
    loop until success:
        response      REST GET snapshot_url
        data      parse response
        last_update_id      data.lastUpdateId
        order_book      data.bids, data.asks
```

Bước 2: Lắng nghe WebSocket và cập nhật Order Book

Sau khi có snapshot, hệ thống sẽ lắng nghe luồng dữ liệu delta từ WebSocket, liên tục cập nhật order book bằng các lệnh mới.

Điều kiện kiểm tra:

- Nếu $U >$ last_update_id $+ 1$: mất dữ liệu, phải tải lại snapshot.
- Nếu $u \geq$ last_update_id: cập nhật order book và lưu lại bản ghi.

Listing 3.2: Lắng nghe WebSocket

```
function listen_order_book():
    connect to WebSocket
    loop:
        message    receive WebSocket
        U, u, bids, asks    parse(message)

        if U > last_update_id + 1:
            get_initial_order_book()
            continue

        if u >= last_update_id:
            update_order_book(bids, asks)
            last_update_id    u
            save_to_buffer(order_book)
```

Bước 3: Lưu dữ liệu theo batch

Thay vì ghi từng dòng vào file CSV, dữ liệu được lưu vào một buffer và chỉ ghi sau mỗi `BATCH_SIZE` bản ghi để tối ưu hiệu suất ghi đĩa.

Listing 3.3: Buffer ghi CSV

```
function buffer_data():
    buffer.append(current_orderbook_snapshot)
    if len(buffer)    BATCH_SIZE:
        save_to_csv(buffer)
        buffer.clear()
```

Bước 4: Chuyển dữ liệu CSV → Parquet

Sau khi có file CSV chứa dữ liệu dạng:

```
record_id | timestamp | lastUpdateId | bids | asks
```

Ta chuyển đổi thành nhiều file Parquet nén Snappy để:

- Nhanh hơn khi load vào RAM.
- Ít chiếm dung lượng đĩa.
- Phù hợp với xử lý số lượng lớn snapshot sau này.

Listing 3.4: Chuyển CSV sang Parquet

```
function convert_csv_to_parquet(csv_file, chunk_size):
    split CSV into chunks of size chunk_size
    for each chunk:
        write chunk to Parquet with snappy compression
```

Bước 5: Gộp lại thành một file Parquet lớn

Dữ liệu từ nhiều file nhỏ được stream batch và gộp lại thành một file duy nhất, giúp xử lý nhanh hơn ở các bước clustering, payoff, hay huấn luyện RL.

Listing 3.5: Gộp các file Parquet

```
function merge_parquet_chunks(input_dir, output_file):
    file_list      sorted list of chunk_*.parquet
    buffer      []
    for each file in file_list:
        for each batch in file:
            buffer.append(batch)
            if len(buffer)      threshold:
                write buffer to output_file
                clear buffer
    if buffer not empty:
        write remaining to output_file
```

3.2.4 Minh hoạ dữ liệu đầu ra

Ví dụ một bản ghi snapshot sau khi chuẩn hoá:

Listing 3.6: Ví dụ một snapshot

```
{
  "record_id": 235,
  "timestamp": "2025-05-26 10:00:01.547",
  "lastUpdateId": 987654321,
  "bids": [["68250.21", "0.8"], ["68248.00", "1.2"], ...],
  "asks": [["68255.00", "0.6"], ["68256.10", "1.5"], ...]
}
```

Cả `bids` và `asks` đều là list các lệnh giới hạn gồm [giá, khối lượng], với các mức sâu hơn level 1.

3.2.5 Đảm bảo tính liên tục và độ tin cậy

- Nếu WebSocket ngắt kết nối, hệ thống sẽ tự động reconnect sau 5 giây.
- Nếu phát hiện dữ liệu mất gói ($U >$ last_update_id $+ 1$), sẽ tự động tải lại snapshot từ REST.
- Dữ liệu được đánh số `record_id` liên tục để tiện kiểm tra, huấn luyện RL theo thời gian.

Tổng kết

Phần này đã xây dựng một pipeline thu thập dữ liệu order book thời gian thực từ Binance:

- Kết hợp REST + WebSocket đảm bảo tính toàn vẹn dữ liệu.
- Ghi dữ liệu hiệu quả với buffer, định dạng chuẩn Parquet.
- Chuẩn bị dữ liệu đầu vào cho các bước tiếp theo: clustering, nhận diện chiến lược bot, và RL agent training.

3.3 Phân cụm các lệnh (Order Clustering)

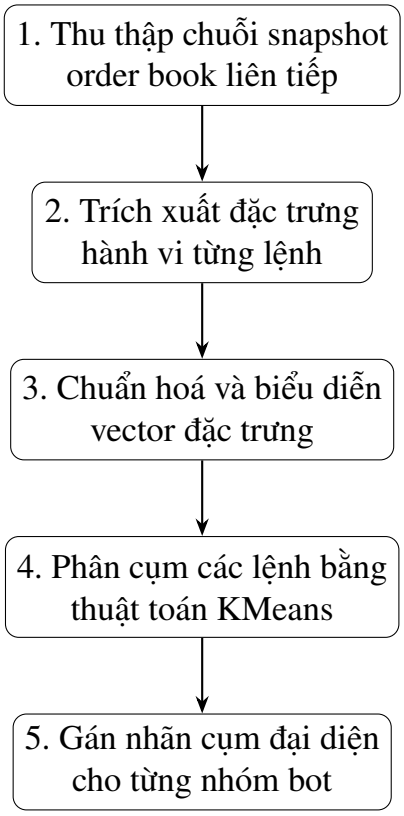
Mục tiêu

Trong thị trường với nhiều tác nhân tham gia, việc nhận diện các chiến lược của các *market makers* là yếu tố then chốt để phân tích hành vi chiến lược và xây dựng mô hình tương tác. Mỗi lệnh trong order book có thể phản ánh hành vi của một tác nhân cụ thể. Tuy nhiên, không thể quan sát trực tiếp danh tính của các tác nhân này. Do đó, bước đầu tiên là **phân cụm các lệnh theo hành vi tương đồng**, từ đó suy đoán ra các nhóm tác nhân.

Ý tưởng

Mỗi lệnh trong order book được đặc trưng bởi nhiều thuộc tính hành vi như: thời gian tồn tại (*lifetime*), khoảng cách đến giá giữa (*log-distance to midprice*), khối lượng, số lần cập nhật, v.v. Ta giả định rằng các lệnh đến từ cùng một Market Maker sẽ có đặc trưng hành vi tương đồng và được duy trì nhất quán qua nhiều snapshot liên tiếp. Việc áp dụng thuật toán phân cụm (ví dụ: KMeans) lên các đặc trưng này sẽ giúp chia các lệnh thành các nhóm đại diện cho các bot khác nhau.

Pipeline tổng quan



Hình 3.2: Pipeline phân cụm các lệnh theo hành vi

Pipeline gồm các bước chính:

1. Thu thập chuỗi snapshot order book liên tiếp.
2. Trích xuất các đặc trưng hành vi của từng lệnh.
3. Chuẩn hoá và biểu diễn vector đặc trưng cho từng lệnh.
4. Áp dụng thuật toán KMeans để phân cụm các lệnh.
5. Gán nhãn cụm cho từng lệnh, đại diện cho từng nhóm bot.

Thuật toán chi tiết

Đầu vào: Danh sách T snapshot liên tiếp của order book, mỗi snapshot chứa thông tin bid/ask.

Đầu ra: Bảng đặc trưng các lệnh trong snapshot cuối, kèm nhãn phân cụm (cluster).

Algorithm 2: Phân cụm các lệnh từ chuỗi snapshot order book

Input: Danh sách snapshot $\mathcal{S} = \{S_1, S_2, \ldots, S_T\}$; số cụm K; số lệnh top N mỗi phía.

Output: Tập đặc trưng các lệnh trong S_T với nhãn phân cụm.

Khởi tạo từ điển $order_tracker$

for $t = 1$ *to* T **do**

Tính $midprice_t = \frac{best_bid + best_ask}{2}$.

for *mỗi lệnh* $(price, volume)$ *trong top-*N *bid/ask của* S_t **do**

Cập nhật lifetime, update_count, volume history vào $order_tracker[(price, side)]$.

Trích xuất đặc trưng cho các lệnh trong snapshot S_T:

for *mỗi lệnh* $o_i = (price_i, volume_i, side_i)$ **do**

Tính $log_distance_i = \log(1 + |price_i - midprice_T|)$.

Tính $log_volume_i = \log(1 + volume_i)$.

Tính $lifetime_i$, $update_count_i$ từ $order_tracker$.

Tính các đặc trưng thị trường: spread, imbalance, bid/ask retention,...

Ghép thành vector đặc trưng $\mathbf{x}_i$.

Gom toàn bộ $\mathbf{x}_i$ thành ma trận X.

Áp dụng KMeans(X, K) $\rightarrow$ Nhãn phân cụm $\{c_i\}$.

Trả về $(\mathbf{x}_i, c_i)$ cho mỗi lệnh.

Các đặc trưng hành vi

Các đặc trưng được thiết kế để phản ánh hành vi của từng lệnh và trạng thái thị trường tại thời điểm hiện tại:

Tên đặc trưng	**Mô tả**
log_distance	Khoảng cách log tới midprice
log_volume	Khối lượng log
lifetime	Số lần xuất hiện liên tiếp qua snapshot
update_count	Số lần bị cập nhật volume
is_static	Lệnh giữ nguyên volume
side	0: bid, 1: ask
rank_norm	Vị trí lệnh / N
spread	Chênh lệch giá bid/ask tốt nhất
bid_ask_ratio	Tỉ lệ volume bid/ask
imbalance	Mức mất cân bằng phía mua và bán
bid/ask_retention	Tỉ lệ lệnh được giữ lại qua snapshot
bid/ask_update_rate	Tần suất thay đổi lệnh
mid_change	Biến động midprice trong chuỗi
volatility	Độ lệch chuẩn midprice

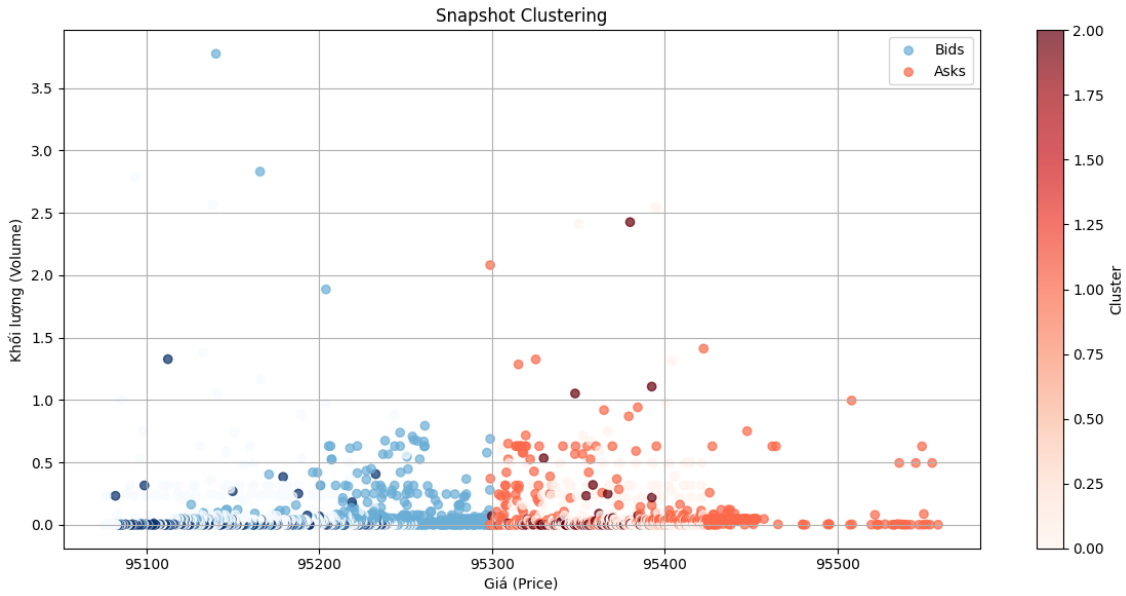
Bảng 3.1: Danh sách các đặc trưng hành vi được sử dụng

Lựa chọn thuật toán phân cụm

Thuật toán KMeans được chọn vì:

- Hiệu quả và dễ cài đặt.
- Có thể điều chỉnh số cụm trước (K), phù hợp với giả định có số lượng bot hữu hạn.
- Hoạt động tốt khi các cụm có hình dạng cầu và độ phân tách rõ ràng.

Minh hoạ



Hình 3.3: Ví dụ minh hoạ phân cụm lệnh trên Orderbook Snapshot

Kết luận

Phân cụm các lệnh theo hành vi là bước đầu tiên để nhận diện các Market Maker ẩn danh trong order book. Việc xây dựng đặc trưng chi tiết và áp dụng thuật toán phân cụm giúp tách biệt các chiến lược khác nhau. Đầu ra của bước này là nền tảng cho các chương sau, nơi mỗi cụm sẽ được nhận diện chiến lược và tham gia vào trò chơi chiến lược Nash.

3.4 Nhận diện chiến lược từng bot

3.4.1 Mục tiêu

Sau khi thực hiện phân cụm các lệnh từ sổ lệnh (order book) theo hành vi tương đồng, phần tập trung vào việc:

- Phân tích các đặc trưng định lượng trung bình của từng cụm.
- Gán nhãn loại bot phù hợp: **MM (Market Maker)**, **TF (Taker Follower)** hoặc **AT (Aggressive Trader)**.
- Nhận diện chiến lược cụ thể của từng bot (ví dụ: *narrow spread*, *sniping buy*, *passive sell*).
- Trích xuất các tham số đặc trưng như spread trung bình, khối lượng đặt lệnh tương đối, độ thiên lệch mua/bán,... để phục vụ cho việc tính toán payoff và mô hình hóa trò chơi ở các chương sau.

3.4.2 Tóm tắt đặc trưng của mỗi cụm

Mỗi cụm (cluster) được xem như đại diện cho một bot cụ thể. Từ đó, các đặc trưng sau được tính toán trung bình cho từng cụm:

- **avg_spread**: mức spread trung bình của các lệnh trong cụm.
- **avg_volume_pct**: tỷ lệ khối lượng đặt lệnh (log-volume) đã được chuẩn hóa.
- **avg_lifetime**: tuổi thọ trung bình của lệnh (đơn vị giây).
- **update_freq**: tần suất cập nhật vị trí lệnh.
- **static_ratio**: tỷ lệ lệnh không cập nhật (passive).
- **aggressiveness**: mức độ chủ động tiếp cận giá khớp lệnh (càng cao càng gần best bid/ask).
- **side_ratio**: tỷ lệ lệnh ở phía ask (giá bán); giá trị > 0.5 cho thấy thiên lệch về bên bán.

3.4.3 Gán nhãn loại bot

Dựa vào các đặc trưng trung bình của từng cụm, mỗi cụm được ánh xạ vào một trong ba loại bot chính theo heuristic rule:

- **AT (Aggressive Trader)**: có độ **aggressiveness rất cao** (> 0.85) và cập nhật thường xuyên ($update_freq > 3$).
- **MM (Market Maker)**: tỷ lệ lệnh **tĩnh cao** ($static_ratio > 0.8$) và spread thấp.
- **TF (Taker Follower)**: không quá chủ động, có thể thiên lệch về một phía và cập nhật linh hoạt.

Việc phân loại giúp xác định vai trò chiến lược trong trò chơi vi mô thị trường.

3.4.4 Nhận diện chiến lược cụ thể

Dựa trên loại bot, chiến lược cụ thể được ánh xạ cùng với các tham số:

- **spread**: mức spread trung bình.
- **volume_pct**: tỷ lệ khối lượng đặt lệnh.
- **aggressiveness**: mức độ chủ động với giá thị trường.
- **lifetime**: thời gian sống trung bình của lệnh.
- **update_count**: số lần cập nhật lệnh trung bình.

Đây là các tham số đầu vào cho mô phỏng khớp lệnh, tính payoff và phân tích cân bằng chiến lược ở các chương tiếp theo.

3.4.5 Thuật toán nhận diện chiến lược

Algorithm 3: Nhận diện chiến lược từng bot từ cụm lệnh

Input: DataFrame `df_result` chứa các lệnh đã gán nhãn `cluster`
Output: Danh sách các chiến lược bot `strategy_list`
`cluster_summary` ← `summarize_clusters(df_result)`
`strategy_list` ← []
Với mỗi `cluster_id` trong `cluster_summary`
`feature_dict` ← `cluster_summary[cluster_id]`
`bot_type` ← `assign_bot_type(feature_dict)`
`strategy` ← `map_to_strategy(bot_type, feature_dict)`
Thêm `lifetime`, `update_count` vào `strategy`
Thêm {`"type"`: `bot_type`, `"strategy"`: `strategy`} vào `strategy_list`
Return `strategy_list`

Tổng kết

- Mỗi cụm lệnh được xem là đại diện cho một bot cụ thể.
- Các đặc trưng định lượng trung bình được sử dụng để mô tả hành vi đặt lệnh.
- Quy tắc heuristic được áp dụng để gán nhãn loại bot và chiến lược tương ứng.
- Chiến lược được định lượng hóa qua các tham số cụ thể, là đầu vào quan trọng cho việc mô hình hóa payoff và phân tích chiến lược giữa các bot.

3.5 Xây dựng ma trận Payoff và tìm Cân bằng Nash

Sau khi các bot được phân cụm và nhận diện chiến lược cụ thể, bước tiếp theo là xây dựng ma trận phần thưởng (payoff) và tìm điểm cân bằng Nash gần đúng giữa

các chiến lược. Việc này cho phép mô hình xác định phần thưởng tiềm năng mà một chiến lược cụ thể có thể nhận được trong môi trường hiện tại, từ đó hỗ trợ quá trình huấn luyện tác nhân Reinforcement Learning (RL).

3.5.1 Xây dựng hàm Payoff

Mỗi loại bot (**Market Making - MM**, **Trend Following - TF**, **Aggressive Taker - AT**) sẽ có công thức tính payoff riêng, dựa trên các đặc trưng chiến lược nhận diện được. Ngoài ra, còn xét đến tác động tương tác giữa các loại bot với nhau thông qua ma trận `INTERACTION_WEIGHTS`.

Công thức tính payoff

Giá trị payoff của mỗi bot được xác định như sau:

- **Bot MM**:

$$\text{payoff}_i = \frac{w_s}{1 + \text{spread}_i} + w_v \cdot \text{volume_pct}_i - w_l \cdot |\text{lifetime}_i - 10|$$

- **Bot TF**:

$$\text{payoff}_i = w_d \cdot \text{distance}_i - w_h \cdot \text{hold_time}_i$$

- **Bot AT**:

$$\text{payoff}_i = w_a \cdot \text{aggressiveness}_i - w_{vol} \cdot \text{volume}_i$$

Trong đó, $w_s, w_v, w_l, w_d, w_h, w_a, w_{vol}$ là các trọng số do người thiết kế hệ thống lựa chọn.

Tác động tương tác giữa các bot

Giả sử bot i là MM và bot j là AT, thì payoff của i còn bị ảnh hưởng bởi hệ số tương tác $\text{interaction}_{i,j} = -20$, được lấy từ bảng tương tác định nghĩa trước.

Pseudocode tính payoff

Algorithm 4: Tính toán hàm payoff tổng hợp

```
compute_payoff_matrix strategy_profiles
Khởi tạo payoff[i] ← 0 cho mọi i
for Với mỗi bot i do
    Xác định type_i và các tham số chiến lược
    Tính payoff[i] dựa trên loại bot và các tham số
    for Với mỗi bot j ≠ i do
        payoff[i] ← payoff[i] + interaction(type_i, type_j)
return payoff
```

3.5.2 Sinh không gian chiến lược

Hàm `strategy_space_fn` được dùng để tạo ra một tập hợp các chiến lược ứng viên (candidate strategies) để từ đó thực hiện tìm kiếm chiến lược tối ưu (best response). Hàm này có thể sử dụng lưới tham số (grid search) kết hợp với sampling ngẫu nhiên để đảm bảo tính đa dạng.

Algorithm 5: Sinh không gian chiến lược cho mỗi loại bot

strategy_space_fnbot_type, $num_samples$
if bot_type == *MM* **then**
Sinh lưới các cặp $(spread, volume_pct, lifetime)$
bot_type == TF
Sinh lưới $(distance, hold_time)$
bot_type == AT
Sinh lưới $(aggressiveness, volume)$
Trả về tập chiến lược ngẫu nhiên từ lưới

3.5.3 Tìm chiến lược phản hồi tốt nhất

Chiến lược phản hồi tốt nhất (Best Response) là chiến lược giúp bot đạt payoff cao nhất, giả sử các bot khác giữ nguyên chiến lược. Trong mỗi vòng lặp, thuật toán sẽ duyệt qua không gian chiến lược và chọn ra ứng viên tốt nhất theo hàm payoff.

Algorithm 6: Tìm phản hồi tốt nhất cho một bot

find_best_responsebot_id, $profiles$, $strategy_space_fn$
Lấy $type$ của bot và sinh $candidates$ từ strategy space
for *mỗi* $candidate$ **do**
Thử cập nhật $profiles[bot_id] \leftarrow candidate$
Tính $payoff$ mới
if $payoff[bot_id] > best_payoff$ **then**
Cập nhật $best_strat \leftarrow candidate$
return $best_strat$

3.5.4 Thuật toán Best Response Dynamics

Dựa trên phản hồi tốt nhất của mỗi bot, ta triển khai thuật toán lặp để hội tụ về một trạng thái mà không bot nào có thể cải thiện payoff bằng cách thay đổi chiến lược của riêng mình — đây là một điểm cân bằng Nash gần đúng.

Algorithm 7: Best Response Dynamics

best_response_dynamics$strategy_profiles$, $strategy_space_fn$
for $t = 1$ ***to*** max_iter **do**
$no_change \leftarrow True$
for *mỗi* bot_id **do**
$best_strat \leftarrow$ find_best_responsebot_id, profiles
if *Chiến lược thay đổi* **then**
Cập nhật chiến lược và đặt $no_change \leftarrow False$
if no_change **then**
break
return $strategy_profiles$

Ý nghĩa

Việc tìm được điểm cân bằng Nash giữa các bot không chỉ giúp hiểu rõ hành vi chiến lược của thị trường, mà còn cung cấp phần thưởng tiềm năng (proxy reward) cho agent RL. Khi agent học theo chiến lược tiệm cận điểm cân bằng, nó sẽ hành động hợp lý và có thể sinh lợi trong môi trường cạnh tranh thực tế.

3.6 Quá trình huấn luyện tác nhân DDPG trong môi trường Market Making

Quá trình huấn luyện tác nhân (agent) DDPG được thực hiện trong một môi trường mô phỏng thị trường gần với dữ liệu thực tế. Môi trường này bao gồm nhiều thành phần phân tích phức tạp như phân cụm hành vi các bot, nhận diện chiến lược, tính toán payoff và tìm kiếm cân bằng Nash tại mỗi bước thời gian. Dưới đây là từng bước của pipeline huấn luyện tác nhân một cách chi tiết:

3.6.1 Sinh quan sát (Observation)

Tại mỗi bước thời gian t, tác nhân RL cần nhận một trạng thái (state) đầu vào thể hiện bối cảnh thị trường hiện tại. Vector quan sát này được xây dựng từ ba nhóm thông tin chính:

- **Thông tin chiến lược hiện tại của tác nhân**:
  - `spread`: mức chênh lệch giá giữa bid và ask mà tác nhân đang áp dụng.
  - `volume_pct`: phần trăm khối lượng lệnh so với tổng thanh khoản hiện có.
  - `inventory`: vị thế hiện tại (long/short) của tác nhân.
  - `step_progress`: tiến trình của episode hiện tại, chuẩn hóa từ 0 đến 1.
- **Thông tin thị trường từ snapshot order book gần nhất**:
  - `midprice`: giá giữa của best bid và best ask (chuẩn hóa log).
  - `spread_book`: spread thực tế của thị trường tại thời điểm đó.
  - `imbalance`: chênh lệch thanh khoản giữa phía mua và bán.
  - `bid_depth_1`, `ask_depth_1`: độ sâu thanh khoản ở mức giá đầu tiên của bid/ask.
- **Thông tin hành vi của các bot market maker khác (đã phân tích)**:
  - `avg_spread_MM`: spread trung bình của các bot dạng Market Maker.
  - `avg_volume_TF_buy`, `avg_volume_TF_sell`: volume trung bình của các bot dạng Trend Following theo hai chiều.
- **Thông tin lịch sử phần thưởng**:
  - `reward`: phần thưởng tại bước trước.
  - `nash_dist`: khoảng cách giữa chiến lược agent và chiến lược Nash ở bước trước.

Tất cả thông số trên được chuẩn hóa về khoảng $[0, 1]$ hoặc $[-1, 1]$ để đảm bảo ổn

định khi đưa vào mạng nơ-ron.

3.6.2 Tác nhân chọn hành động (Action Selection)

Tác nhân sử dụng mô hình **Deep Deterministic Policy Gradient (DDPG)** để sinh hành động liên tục trong không gian hành động 3 chiều:

- `bid_spread`: mức chênh lệch phía mua.
- `ask_spread`: mức chênh lệch phía bán.
- `volume_pct`: phần trăm volume đặt lệnh so với tổng thanh khoản.

Tác nhân sẽ lấy quan sát hiện tại làm đầu vào cho policy network, từ đó sinh ra một hành động cụ thể tại bước thời gian t, thể hiện chiến lược giao dịch được đề xuất trong trạng thái thị trường hiện tại.

3.6.3 Mô phỏng thị trường và tính toán phần thưởng (Reward Computation)

Phần thưởng không được xác định đơn giản dựa trên PnL mà dựa trên khái niệm payoff trong trò chơi giữa các bot.

Cụ thể:

1. Từ snapshot mới nhất và hành động của tác nhân ở bước trước, tiến hành:
   - **Phân cụm các lệnh** trong order book bằng hàm `clustering_fn`.
   - **Nhận diện chiến lược** của các bot từ các cluster với `strategy_recognition_fn`.
   - **Tính payoff thực tế** của hành động mà tác nhân đã thực hiện thông qua `compute_payoff_of_action_fn`.
2. **Tính toán cân bằng Nash** (xấp xỉ) bằng cách:
   - Tổng hợp các chiến lược của các bot khác và thêm chiến lược của agent vào tập profile.
   - Áp dụng thuật toán *best-response dynamics* để tìm ra cân bằng Nash thông qua `best_response_dynamics_fn`.
3. **Tính khoảng cách** giữa hành động của tác nhân và chiến lược cân bằng Nash:

$$d = \|a_{t-1}^{agent} - a_{Nash}\|_2$$

4. **Reward cuối cùng** tại bước t được xác định theo công thức:

$$r_t = \text{Payoff}(a_{t-1}) - \lambda \cdot d$$

   với λ là hệ số phạt nếu chiến lược của agent lệch quá xa chiến lược Nash.

Điều này phản ánh rằng agent không chỉ học cách tối đa hóa payoff cá nhân mà còn phải điều chỉnh để tiệm cận với chiến lược hợp lý theo lý thuyết trò chơi (*Game-Theoretic Rationality*).

3.6.4 Cập nhật mô hình và replay buffer

Sau khi thu được tuple:

$$(\text{observation}_t, \text{action}_t, \text{reward}_t, \text{next_observation}_{t+1})$$

Thông tin này được đưa vào bộ nhớ *Replay Buffer*, dùng để:

- Trích xuất mini-batch mẫu hóa trải nghiệm.
- Cập nhật mạng **Actor** và **Critic** của DDPG qua *gradient descent*, nhằm học chính sách tối ưu hóa reward trong dài hạn.

3.6.5 Học tăng cường online liên tục (Online RL Training)

Vì môi trường luôn sử dụng snapshot mới từ dữ liệu thực (hoặc gần thực tế), agent sẽ học được cách:

- Thích nghi với thị trường biến động liên tục.
- Phản ứng với các hành vi thay đổi của những market makers khác.
- Cân bằng giữa lợi nhuận và chiến lược phòng thủ theo lý thuyết trò chơi.

Tổng kết

Mô hình huấn luyện DDPG trong môi trường `MarketMakingEnv` không chỉ đơn thuần là một hệ thống học tăng cường, mà là sự tích hợp chặt chẽ giữa:

- Phân tích hành vi thực tế của các bot,
- Ứng dụng lý thuyết trò chơi (Game Theory),
- Và Chiến lược Market Making với khối lượng nhỏ không làm ảnh hưởng thị trường.

Nhờ vậy, agent không học "vô tội vạ" mà học cách tối ưu hóa chiến lược một cách hợp lý và an toàn trong môi trường cạnh tranh đa tác nhân.

3.7 Huấn luyện mô hình Reinforcement Learning cho Market Making

Sau khi hoàn thiện pipeline xử lý dữ liệu order book thực tế, phân cụm hành vi các bot, nhận diện chiến lược của từng bot và tính toán điểm cân bằng Nash, bước cuối cùng là huấn luyện một mô hình học tăng cường (Reinforcement Learning - RL) để tìm ra chiến lược đặt lệnh tối ưu cho một agent Market Making.

Mô hình học theo phong cách **real-time**, tức là tại mỗi bước thời gian, agent sẽ nhận vào snapshot dữ liệu thật, tính toán lại toàn bộ chiến lược của các bot đang hoạt động, từ đó xác định phần thưởng dựa trên chiến lược Nash trước đó.

3.7.1 Tải dữ liệu order book thực tế từ file Parquet

Mục tiêu: Xây dựng một loader thời gian thực từ file `.parquet`, cung cấp snapshot tương ứng tại mỗi bước của môi trường RL.

Mô tả: Dữ liệu order book được lưu dưới dạng bảng, trong đó mỗi bản ghi (snapshot) có định danh bằng trường `record_id`. Ta xây dựng hàm `make_data_loader_fn()` để tạo ra một hàm `loader(step)` cho phép truy xuất snapshot theo bước thời gian.

Pseudocode:

Algorithm 8: Tạo hàm đọc snapshot từ file Parquet

```
make_data_loader_fnfile_path
dataset ← load Parquet file using pyarrow
snapshot_ids ← sorted list of unique record_id
loaderstep
idx ← min(step, len(snapshot_ids) - 1)
snapshot_id ← snapshot_ids[idx]
snapshot ← filter dataset by record_id = snapshot_id
return snapshot.to_dataframe(), snapshot_id
return loader, len(snapshot_ids)
```

3.7.2 Khởi tạo môi trường RL: `MarketMakingEnv`

Mục tiêu: Khởi tạo môi trường tương tác để mô hình RL tương tác với snapshot thực tế, thực hiện clustering, nhận diện chiến lược bot và tính toán payoff matrix.

Mô tả: Môi trường tuân theo chuẩn Gym, được thiết kế để mỗi bước sẽ lấy snapshot mới, gọi pipeline clustering-strategy-payoff và phản hồi trạng thái 16 chiều, phần thưởng theo chiến lược Nash gần nhất.

Pseudocode:

Algorithm 9: Khởi tạo môi trường huấn luyện RL

```
init_environment
env ← MarketMakingEnv(
    agent_type = "MM",
    lambda_penalty = 1000,
    data_loader_fn = make_data_loader_fn(),
    clustering_fn = cluster_orders(),
    strategy_recognition_fn = recognize_strategy(),
    compute_payoff_matrix_fn = compute_payoff_matrix(),
    best_response_dynamics_fn = BR_dynamics(),
    compute_payoff_of_action_fn = compute_payoff_of_action() )
return env
```

3.7.3 Thuật toán huấn luyện DDPG

Mục tiêu: Sử dụng Deep Deterministic Policy Gradient để huấn luyện agent trong không gian hành động liên tục.

Pseudocode:

Algorithm 10: Thuật toán huấn luyện DDPG

```
for episode in 1 to max_episodes do
state ← env.reset()
for step in 1 to max_steps do
action ← actor(state) + exploration_noise
next_state, reward, done ← env.step(action)
buffer.add(state, action, reward, next_state, done)
if len(buffer) > batch_size then
batch ← buffer.sample(batch_size)
update_critic(batch)
update_actor(batch)
state ← next_state
if done then
break
```

3.7.4 Tính phần thưởng từ chiến lược Nash

Mô tả: Phần thưởng tại mỗi bước là:

$$\text{reward} = \text{payoff(action)} - \lambda \cdot \text{distance}(action, \text{Nash})$$

Pseudocode:

Algorithm 11: Tính reward theo khoảng cách đến chiến lược Nash

```
compute_rewardaction, strategy_space, nash_strategy
payoff ← compute_payoff_of_action(action, strategy_space)
distance ← compute_distance_to_strategy(action, nash_strategy)
reward ← payoff - lambda * distance
return reward
```

3.7.5 Lưu mô hình định kỳ

Mục tiêu: Lưu trọng số mô hình mỗi 10 vòng lặp để theo dõi và khôi phục.

Pseudocode:

Algorithm 12: Lưu checkpoint

```
if episode % 10 == 0 then
save_model(actor, "actor.pth")
save_model(critic, "critic.pth")
```

3.8 Đánh giá chiến lược RL bằng mô phỏng khớp lệnh

Sau khi huấn luyện xong agent DDPG để đóng vai trò một Market Maker, bước đánh giá (Inference & Backtest) sẽ thực hiện như sau:

1. Agent chọn hành động (spread, volume) tại mỗi snapshot.
2. Mô phỏng khớp lệnh hành động agent với sổ lệnh thực tế.
3. Tính toán lợi nhuận (PnL) và các chỉ số đánh giá (Sharpe ratio).

Bước 1: Tải dữ liệu sổ lệnh và khởi tạo môi trường

Sử dụng dữ liệu order book snapshot theo định dạng Parquet. Mỗi snapshot chứa thông tin về các mức giá và khối lượng bid/ask hiện có. Môi trường RL thực hiện toàn bộ pipeline: phân cụm bot, nhận diện chiến lược, tính toán Nash Equilibrium, tính payoff,...

Bước 2: Agent chọn hành động

Tại mỗi bước t, agent sẽ chọn hành động:

- `bid_spread` $\in [0, 0.01]$: mức giảm giá so với midprice khi đặt lệnh mua.
- `ask_spread` $\in [0, 0.01]$: mức tăng giá so với midprice khi đặt lệnh bán.
- `volume_pct` $\in [0, 1]$: phần trăm khối lượng muốn giao dịch.

Bước 3: Mô phỏng khớp lệnh

Agent không đặt lệnh thực sự mà mô phỏng khớp lệnh với các mức giá có sẵn trong sổ lệnh thực tế.

Giả thiết:

- Agent bán ra tại $ask_price = midprice \times (1 + ask_spread)$
- Agent mua vào tại $bid_price = midprice \times (1 - bid_spread)$

Nếu giá này tốt hơn hoặc bằng giá thị trường (có người sẵn sàng giao dịch), thì sẽ khớp lệnh tương ứng.

Pseudocode: simulate_matching

```
Input: action = (bid_spread, ask_spread, volume_pct), snapshot,
midprice
Output: danh sách trades đã thực hiện

1. Tính giá bid và ask của agent:
     bid_price = midprice × (1 - bid_spread)
     ask_price = midprice × (1 + ask_spread)
```

```
     total_volume = volume_pct × midprice

2. Duyệt qua bên BID trong order book:
     Nếu price  ask_price → khớp lệnh SELL
     Ghi nhận trade, giảm vol_remain

3. Duyệt qua bên ASK trong order book:
     Nếu price  bid_price → khớp lệnh BUY
     Ghi nhận trade, giảm vol_remain

4. Trả về danh sách trade đã khớp
```

Bước 4: Tính lợi nhuận PnL

Tính lợi nhuận từ các giao dịch đã khớp:

- **Cash**: tổng tiền ra vào do giao dịch
- **Inventory**: lượng tài sản hiện còn nắm giữ

Nếu còn tồn kho, giả sử thanh lý ở giá trung bình để ước lượng PnL.

Công thức PnL:

$$\text{PnL} = \text{Cash} + \text{Inventory} \times \text{MidPrice}$$

Pseudocode: Compute Pnl

```
Input: trades = danh sách buy/sell
Output: PnL

1. Khởi tạo cash = 0, inventory = 0

2. Duyệt qua mỗi trade:
    Nếu BUY → inventory += vol, cash -= price × vol
    Nếu SELL → inventory -= vol, cash += price × vol

3. Nếu inventory  0:
    Giá trị tồn kho = inventory × mid_price trung bình
    Cộng vào cash

4. Trả về cash như là PnL
```

Bước 5: Tính hệ số Sharpe

Sharpe đo hiệu quả chiến lược so với rủi ro, chuẩn hóa theo số ngày giao dịch:

Công thức Sharpe:

$$\text{Sharpe Ratio} = \frac{\mu_R}{\sigma_R} \times \sqrt{252}$$

Trong đó:

- μ_R là lợi nhuận trung bình,
- σ_R là độ lệch chuẩn lợi nhuận,
- 252 là số phiên giao dịch ước tính trong năm.

Pseudocode: Compute Sharpe

```
Input: pnl_history
Output: Sharpe Ratio

1. Tính returns = pnl_history[t] - pnl_history[t-1]

2. Nếu số returns < 2 → trả về 0

3. Tính mean_return = trung bình của returns
         std_return = độ lệch chuẩn của returns

4. Sharpe = mean_return / std_return × sqrt(252)

5. Trả về Sharpe
```

Minh họa khớp lệnh

- midprice = 100.00
- bid_spread = 0.002 ⇒ bid_price = 99.80
- ask_spread = 0.002 ⇒ ask_price = 100.20
- volume_pct = 0.01 ⇒ volume = 1.0

Nếu có:

- Bid levels: (100.30, 0.5), (100.25, 1.0)
- Ask levels: (99.70, 0.3), (99.60, 0.7)

⇒ Agent sẽ khớp:

- SELL tại 100.30 (0.5), 100.25 (0.5) ⇒ tổng 1.0

- BUY tại 99.70 (0.3), 99.60 (0.7) ⇒ tổng 1.0

Tổng kết pipeline đánh giá

Bước	Thành phần	Mô tả
1	`env.reset()`	Lấy snapshot đầu tiên
2	`agent.act(state)`	Chọn hành động dựa vào quan sát
3	`env.step(action)`	Cập nhật snapshot mới và chiến lược thị trường
4	`simulate_matching()`	Mô phỏng khớp lệnh hành động agent
5	`compute_pnl()`	Tính lợi nhuận từ khớp lệnh
6	`compute_sharpe()`	Tổng hợp đánh giá chiến lược

Bảng 3.2: Tổng quan pipeline đánh giá chiến lược RL

Kết quả đầu ra

Sau quá trình backtest, các kết quả thu được bao gồm:

- Biểu đồ PnL theo thời gian
- Giá trị Sharpe Ratio

Tổng kết lại, chương này đã trình bày chi tiết toàn bộ quy trình kỹ thuật nhằm hiện thực hóa mô hình Market Making thông minh dựa trên sự kết hợp giữa Học tăng cường sâu và Lý thuyết trò chơi. Từ bước thu thập và xử lý dữ liệu thị trường theo thời gian thực, phân cụm hành vi bot, nhận diện chiến lược, đến mô phỏng trò chơi chiến lược và huấn luyện tác nhân RL – mỗi thành phần trong pipeline đều được xây dựng với mục tiêu phản ánh sát thực trạng thị trường và nâng cao khả năng thích ứng của tác nhân. Mô hình không chỉ học từ dữ liệu mà còn học từ sự tương tác chiến lược với các đối thủ, tạo nên một hệ thống giao dịch tự động có khả năng phản ứng linh hoạt và tối ưu hóa hiệu suất trong môi trường cạnh tranh. Những cơ sở kỹ thuật được trình bày tại chương này sẽ làm nền tảng trực tiếp cho các phân tích thực nghiệm, đánh giá hiệu quả và so sánh chiến lược trong chương tiếp theo.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả thực nghiệm nhằm đánh giá hiệu quả của mô hình Reinforcement Learning tích hợp Game Theory trong bối cảnh thị trường tài chính thực. Các chỉ số đánh giá như lợi nhuận (PnL) và hệ số Sharpe Ratio được sử dụng để đo lường mức độ sinh lời và độ ổn định của chiến lược.

Để kiểm chứng tính hiệu quả, mô hình đề xuất được so sánh với ba phương pháp baseline tiêu biểu trong lĩnh vực giao dịch tài chính bằng học tăng cường. Sự so sánh này cho phép đánh giá toàn diện cả về mặt lý thuyết lẫn thực nghiệm, đồng thời làm rõ những điểm mạnh của mô hình đề xuất trong việc tận dụng thông tin chiến lược từ các tác nhân thị trường thông qua phân tích Game Theory.

4.1 Các tham số đánh giá

Trong quá trình đánh giá hiệu quả của chiến lược Market Making được huấn luyện bằng Reinforcement Learning, hai tham số chính được sử dụng là **Profit and Loss (PnL)** và **Sharpe Ratio**. Đây là những chỉ số tiêu chuẩn trong lĩnh vực tài chính định lượng để đo lường hiệu quả và mức độ rủi ro của một chiến lược giao dịch.

4.1.1 Profit and Loss (PnL)

Chỉ số PnL thể hiện tổng lợi nhuận hoặc thua lỗ của chiến lược giao dịch sau một giai đoạn thực thi. Trong bối cảnh nghiên cứu này, PnL được tính dựa trên dòng tiền thực hiện từ các lệnh giao dịch đã khớp và giá trị tồn kho tại thời điểm kết thúc.

Cụ thể, PnL được xác định như sau:

$$\text{PnL} = \text{Cash} + \text{Inventory} \times \text{MidPrice} \tag{4.1}$$

Trong đó:

- **Cash**: tổng dòng tiền vào/ra do các lệnh mua và bán.
- **Inventory**: khối lượng tài sản còn nắm giữ tại thời điểm đánh giá.
- **MidPrice**: giá trung bình giữa giá mua (bid) và bán (ask) tại thời điểm hiện tại.

Nếu tồn kho chưa được thanh lý, chiến lược giả định rằng toàn bộ lượng tài sản này có thể được quy đổi về tiền mặt theo giá MidPrice hiện tại. Điều này giúp đưa ra một ước lượng thực tế về tổng giá trị mà chiến lược đang nắm giữ.

4.1.2 Sharpe Ratio

Sharpe Ratio là một chỉ số đo lường mức độ hiệu quả của chiến lược giao dịch khi xét đến rủi ro. Nó thể hiện lợi nhuận trung bình thu được trên mỗi đơn vị rủi ro (độ lệch chuẩn lợi nhuận). Trong nghiên cứu này, Sharpe Ratio được chuẩn hóa theo số phiên giao dịch trong một năm, giả định là 252 phiên.

Công thức tính Sharpe Ratio:

$$\text{Sharpe Ratio} = \frac{\mu_R}{\sigma_R} \times \sqrt{252} \tag{4.2}$$

Trong đó:

- μ_R: lợi nhuận trung bình theo từng bước (daily return hoặc bước RL tương đương).
- σ_R: độ lệch chuẩn của các khoản lợi nhuận.
- 252: số phiên giao dịch tiêu chuẩn trong một năm.

Chỉ số Sharpe càng cao chứng tỏ chiến lược càng hiệu quả trên đơn vị rủi ro, và thường được sử dụng để so sánh giữa các chiến lược khác nhau trong cùng một điều kiện thị trường.

4.2 Phương pháp mô phỏng

Trong phần này, chúng tôi trình bày quy trình thiết kế thí nghiệm, cách xử lý dữ liệu, mô hình học tăng cường được triển khai, cũng như các mô hình so sánh (baseline) và tiêu chí đánh giá hiệu quả. Mục tiêu là kiểm chứng tính khả thi và hiệu quả của mô hình Market Making kết hợp giữa Reinforcement Learning và Lý thuyết Trò chơi khi áp dụng vào môi trường thị trường thực tế.

Tập dữ liệu và tiền xử lý

Dữ liệu order book được thu thập từ sàn Binance thông qua WebSocket API, trong khoảng thời gian từ ngày 23/02/2025 đến 06/03/2025, với tổng cộng khoảng 1 triệu snapshot. Tập dữ liệu được chia theo tỷ lệ 80-20 thành:

- **Tập huấn luyện:** khoảng 800.000 snapshot đầu tiên.
- **Tập kiểm thử:** khoảng 200.000 snapshot cuối cùng, được chia thành 8 file định dạng `.parquet`, mỗi file chứa xấp xỉ 25.000 snapshot.

Trước khi đưa vào huấn luyện và đánh giá, dữ liệu được chuẩn hóa và xử lý để đảm bảo độ ổn định đầu vào. Các đặc trưng được trích xuất gồm: `lifetime`, `log_distance`, `log_volume`, `update_count`, cùng nhiều đặc trưng thị trường khác như `spread_book`, `imbalance`, `depth_1`.

Mô hình huấn luyện và tham số

Môi trường huấn luyện là `MarketMakingEnv` do nhóm phát triển, với không gian quan sát gồm 7 chiều bao gồm thông tin về trạng thái agent, đặc trưng thị trường, chiến lược bot khác và lịch sử phần thưởng. Phần thưởng tại mỗi bước được tính theo công thức:

$$\text{reward} = \text{payoff} - \lambda \cdot \text{distance to Nash}$$

với λ là hệ số điều chỉnh mức độ bám sát chiến lược Nash Equilibrium.

Thuật toán học chính được sử dụng trong hệ thống là **Deep Deterministic Policy Gradient (DDPG)**, một phương pháp Actor-Critic phù hợp với không gian hành động liên tục. Các tham số huấn luyện được thiết lập cụ thể như sau:

- **Learning rate:**
  - Actor: 1×10^{-3}
  - Critic: 1×10^{-3}
- **Discount factor:** $\gamma = 0.99$
- **Batch size:** 64
- **Replay buffer:** Lưu trữ tối đa 100.000 bước (transitions)
- **Noise cho exploration:** Thêm nhiễu Gauss $\mathcal{N}(0, 0.5)$ vào hành động
- **Target update rate:** $\tau = 0.005$ (sử dụng cơ chế *soft update* cho các mạng mục tiêu)
- **Action range:** Hành động đầu ra nằm trong khoảng $[-1, 1]$, được nhân với hệ số `action_scale` để chuyển thành giá trị thực tế
- **Hệ số phạt chiến lược:** $\lambda = 1000$, dùng trong hàm phần thưởng để trừ vào khoảng cách giữa chiến lược agent và chiến lược cân bằng Nash gần nhất.

Ngoài ra, trong quá trình huấn luyện, hệ thống còn thực hiện kiểm tra và loại bỏ các giá trị không hợp lệ (`NaN`, `Inf`) nhằm đảm bảo ổn định số và tránh gradient bùng nổ hoặc mất mát thông tin.

Chiến lược của các bot đối thủ được cập nhật liên tục thông qua pipeline gồm: phân cụm lệnh, nhận diện chiến lược, xây dựng ma trận payoff và tìm kiếm chiến lược Nash bằng thuật toán phản hồi tốt nhất lặp (Best Response Dynamics). Agent sẽ học cách tối ưu hành vi của mình bằng cách khai thác sự chênh lệch giữa trạng thái thị trường và chiến lược cân bằng.

4.2.1 Các baseline được sử dụng để so sánh

Trong phần này, chúng tôi lựa chọn ba phương pháp nổi bật từ các nghiên cứu trước đó làm *baseline* để so sánh với mô hình Reinforcement Learning tích hợp Game Theory mà chúng tôi đề xuất. Các baseline này đại diện cho những hướng tiếp cận khác nhau trong giao dịch tài chính bằng học tăng cường, từ phương pháp cổ điển đến kỹ thuật hiện đại sử dụng Deep Learning và Ensemble.

- **Baseline 1: Recurrent Reinforcement Learning (RRL) – Direct Reinforcement (Moody & Saffell, 2001) [4]**
  RRL học chính sách giao dịch thông qua *direct reinforcement* mà không cần mô hình dự báo. Mô hình này trực tiếp tối ưu Sharpe Ratio và được đánh giá là hiệu quả hơn Q-learning trong các kịch bản thực tế.
- **Baseline 2: Deep Reinforcement Learning Ensemble (Yang et al., 2020) [6]**
  Mô hình kết hợp ba thuật toán actor-critic gồm PPO, A2C và DDPG thành một chiến lược giao dịch thống nhất. Phương pháp này cho thấy khả năng thích ứng tốt và đạt Sharpe Ratio cao hơn so với từng thuật toán đơn lẻ.
- **Baseline 3: RL with Technical Indicators (Yasin & Gill, 2022) [2]**
  Phương pháp sử dụng các chỉ báo kỹ thuật làm đầu vào cho tác tử RL nhằm cải thiện quyết định mua/bán. Mặc dù chỉ là một framework cơ bản, mô hình này nhấn mạnh tầm quan trọng của việc xử lý đặc trưng phù hợp trong giao dịch tài chính.

Tất cả các mô hình baseline và mô hình đề xuất được so sánh dựa trên **Sharpe Ratio**, thước đo phản ánh hiệu quả lợi nhuận điều chỉnh theo rủi ro. Các mô hình được huấn luyện và kiểm tra trên cùng tập dữ liệu thị trường với cùng mức phí giao dịch. Kết quả Sharpe Ratio được ghi nhận trung bình qua nhiều lần mô phỏng để đảm bảo độ tin cậy và phản ánh chính xác hiệu suất của từng chiến lược.

4.2.2 Thiết kế thí nghiệm và tiêu chí đánh giá

Mô hình được chạy thử nghiệm toàn bộ tập kiểm thử gồm 8 file `.parquet`, mỗi file bao gồm xấp xỉ 25.000 snapshot.

Các chỉ số đánh giá hiệu suất gồm:

- **PnL:** lợi nhuận thuần sau toàn bộ phiên.
- **Sharpe Ratio:** tỷ lệ giữa lợi nhuận trung bình và độ lệch chuẩn của lợi nhuận.

Phần này trình bày toàn bộ quy trình thiết lập thí nghiệm nhằm đánh giá hiệu quả mô hình được đề xuất. Việc sử dụng dữ liệu thật, kết hợp nhiều baseline đa dạng

và tiêu chí đo lường toàn diện giúp đảm bảo tính khách quan và đầy đủ trong phân tích. Các kết quả chi tiết cùng biểu đồ và bảng thống kê sẽ được trình bày ở phần tiếp theo.

4.3 Kết quả Backtest trên dữ liệu kiểm thử

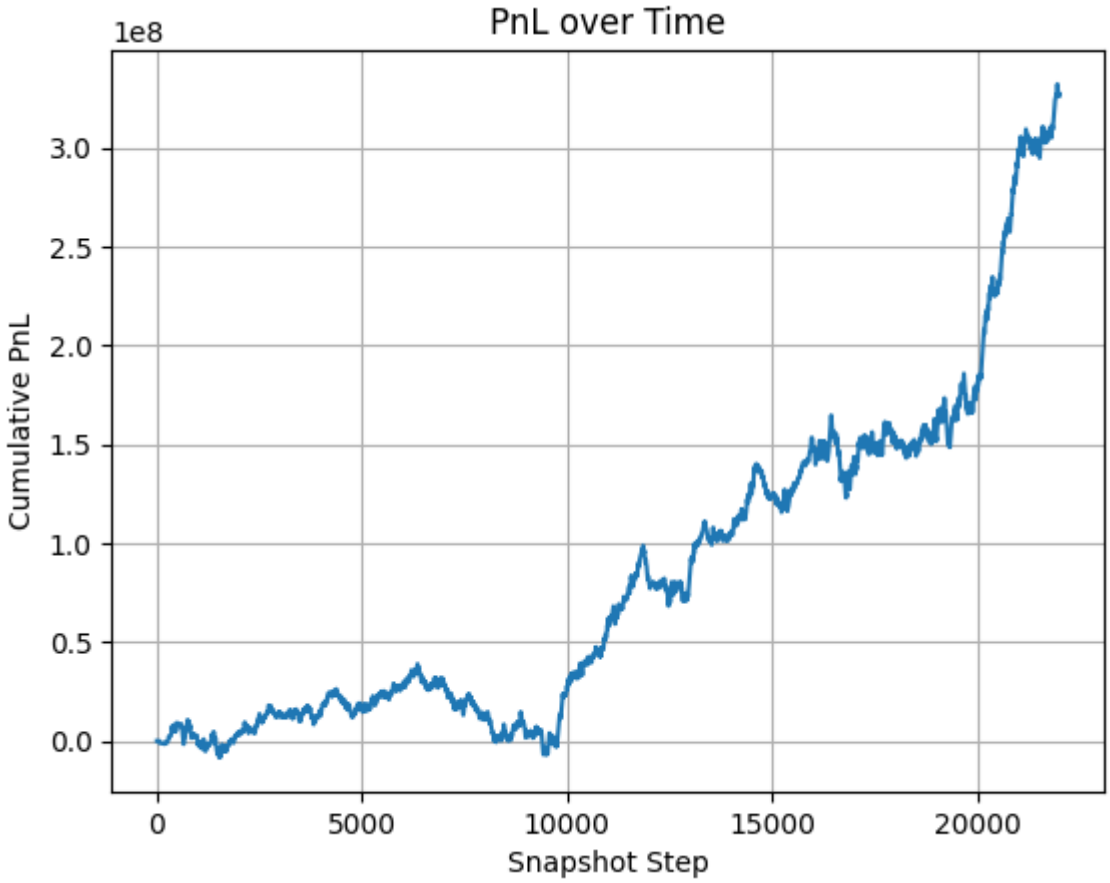
Dưới đây là kết quả thực nghiệm trên 8 bộ dữ liệu kiểm thử, bao gồm biểu đồ PnL và chỉ số Sharpe Ratio thực nghiệm:

Bộ dữ liệu Test 1:

Hình 4.1: Biểu đồ PnL trên Bộ dữ liệu Test 1

Chỉ số Sharpe Ratio: 0.48480166

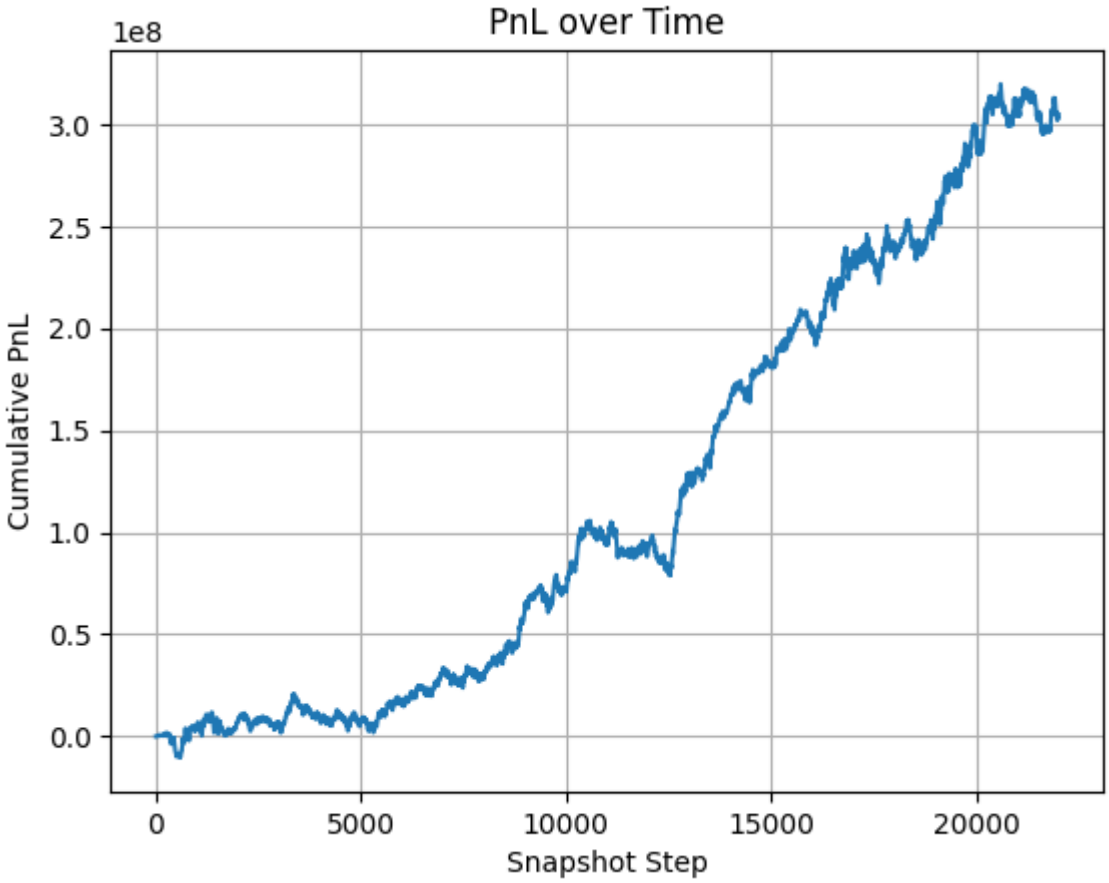
Bộ dữ liệu Test 2:



Hình 4.2: Biểu đồ PnL trên Bộ dữ liệu Test 2

Chỉ số Sharpe Ratio: 1.22428407

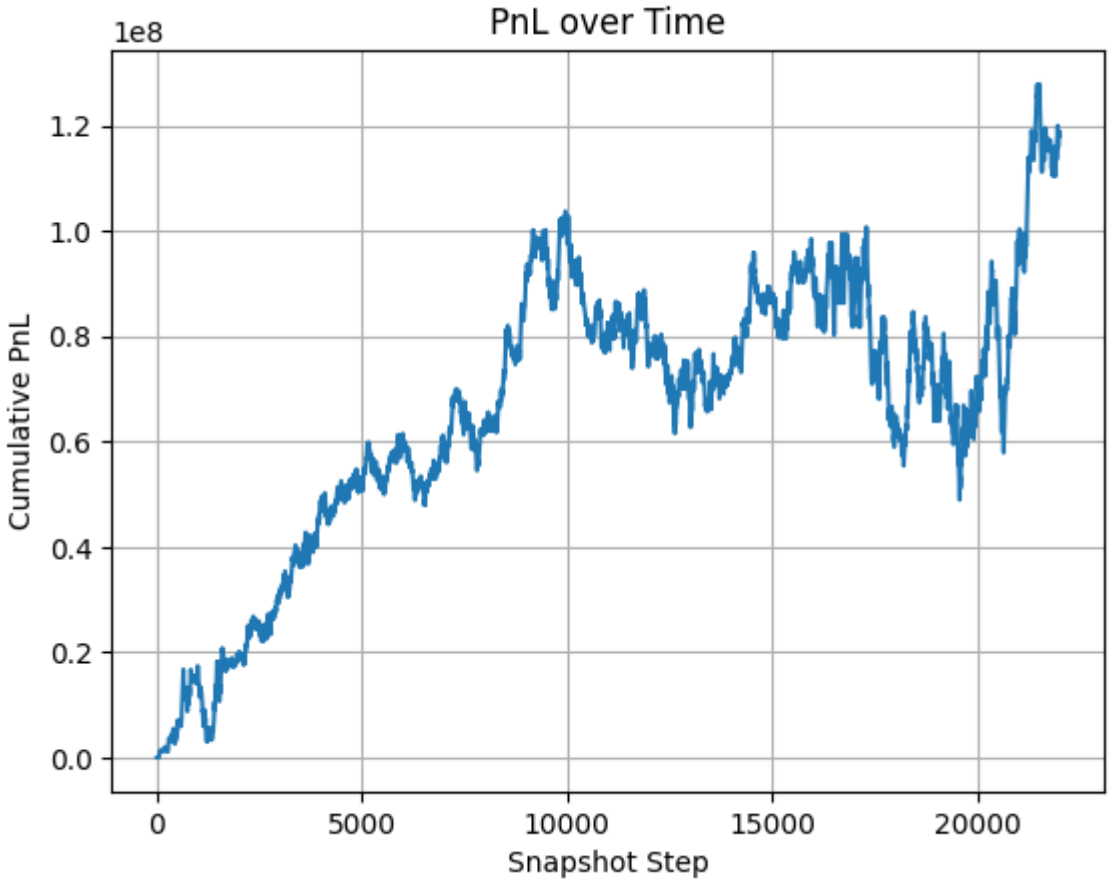
Bộ dữ liệu Test 3:



Hình 4.3: Biểu đồ PnL trên Bộ dữ liệu Test 3

Chỉ số Sharpe Ratio: 1.14535412

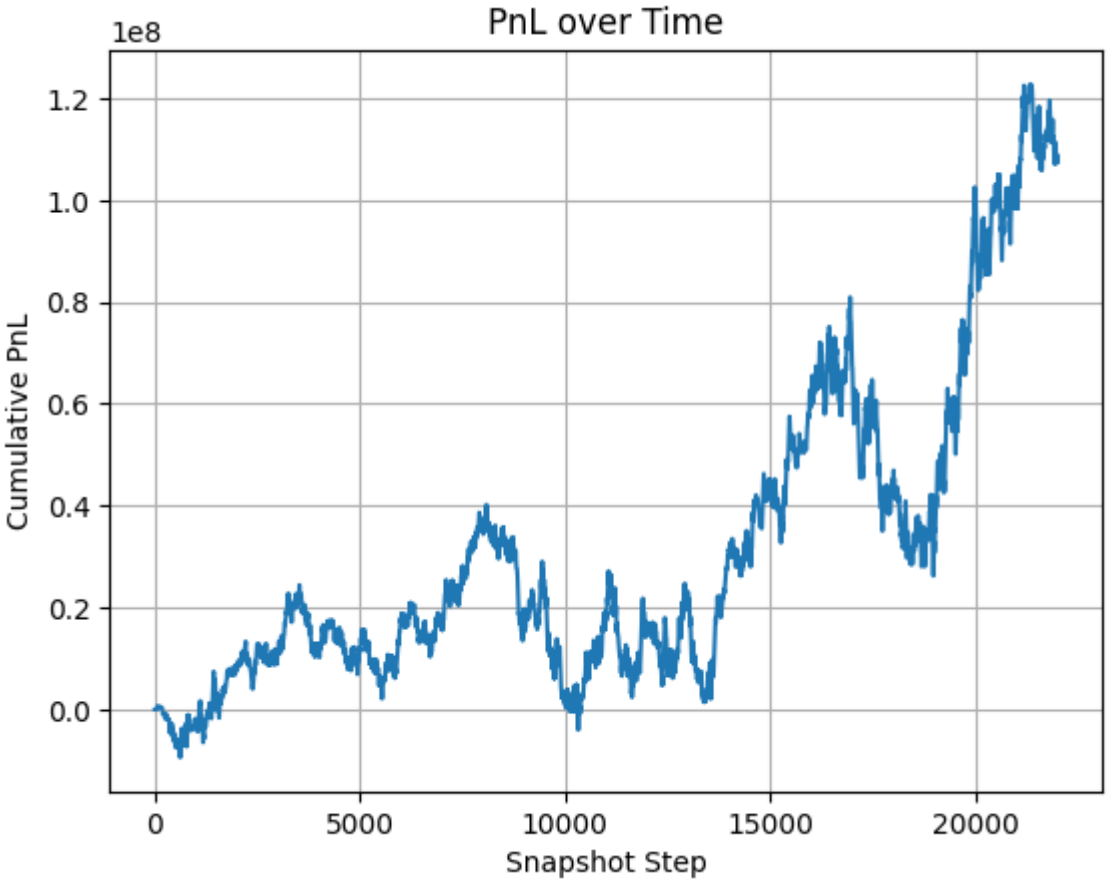
Bộ dữ liệu Test 4:



Hình 4.4: Biểu đồ PnL trên Bộ dữ liệu Test 4

Chỉ số Sharpe Ratio: 0.44032370

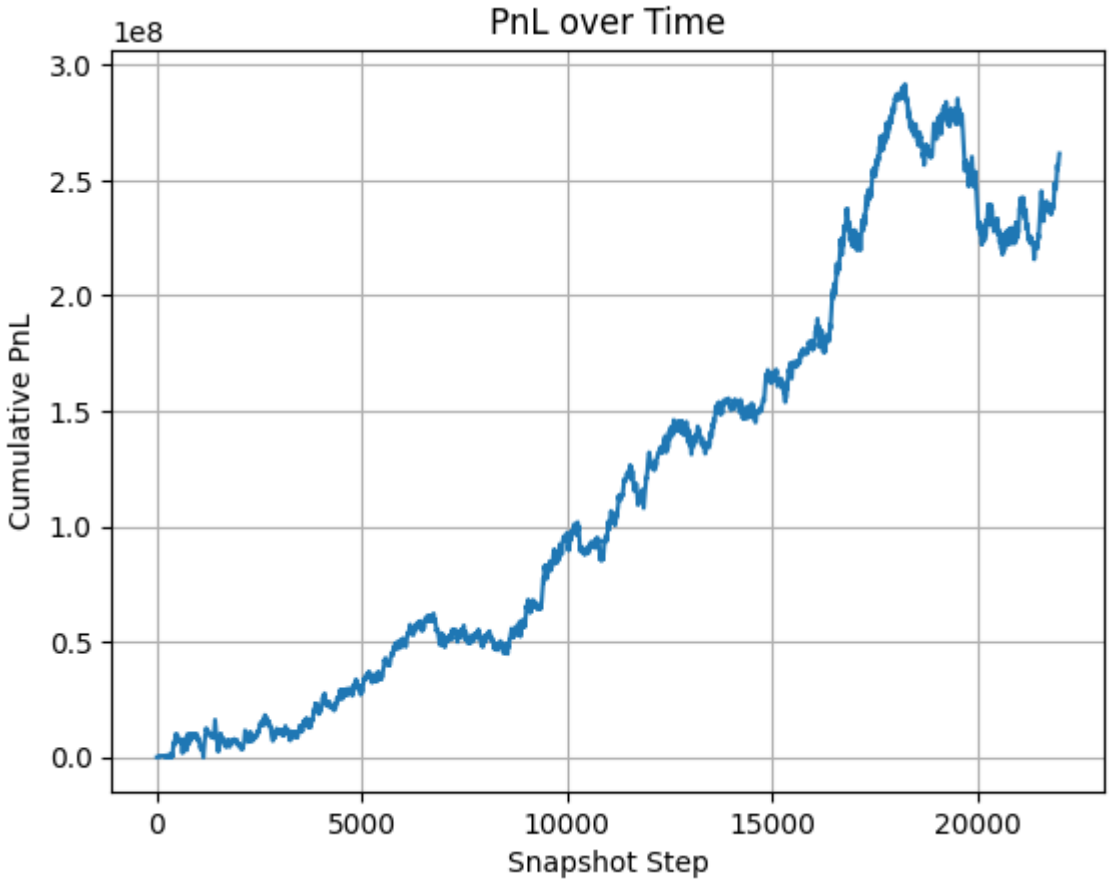
Bộ dữ liệu Test 5:



Hình 4.5: Biểu đồ PnL trên Bộ dữ liệu Test 5

Chỉ số Sharpe Ratio: 0.40523872

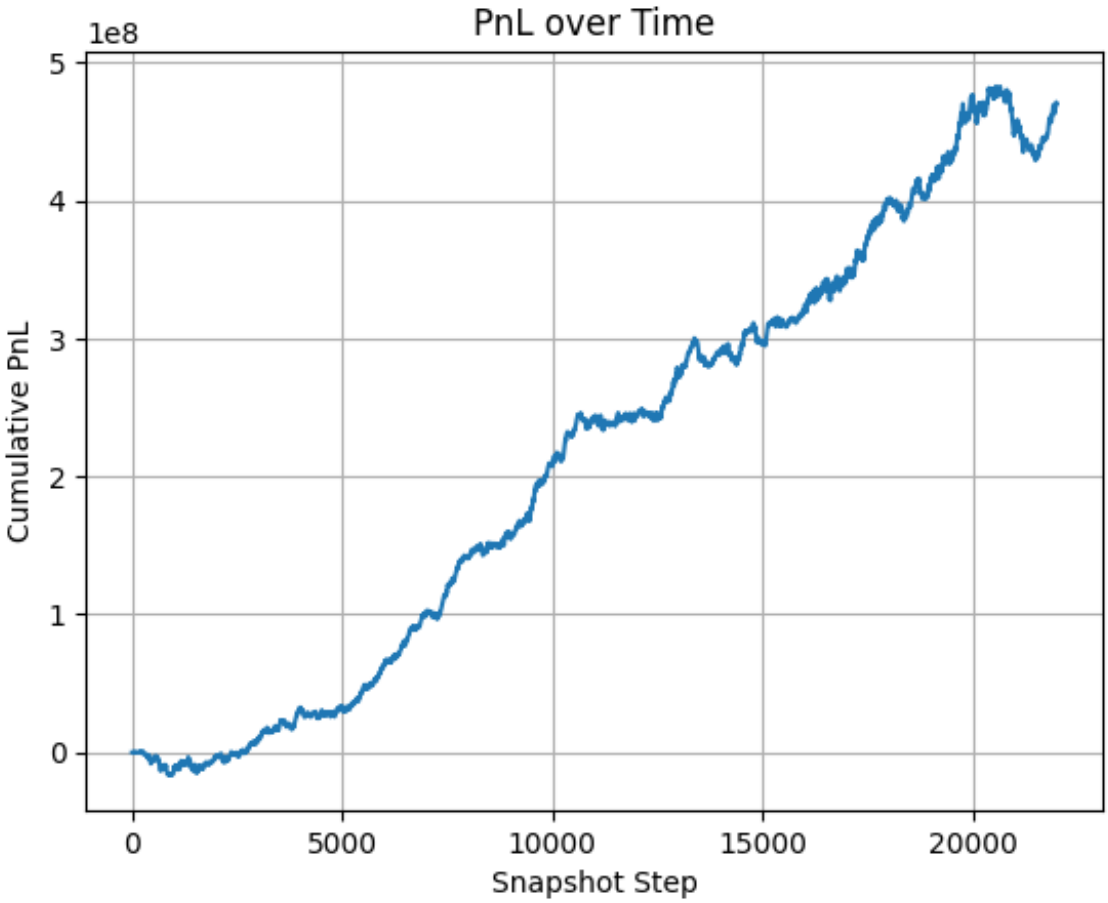
Bộ dữ liệu Test 6:



Hình 4.6: Biểu đồ PnL trên Bộ dữ liệu Test 6

Chỉ số Sharpe Ratio: 0.98056983

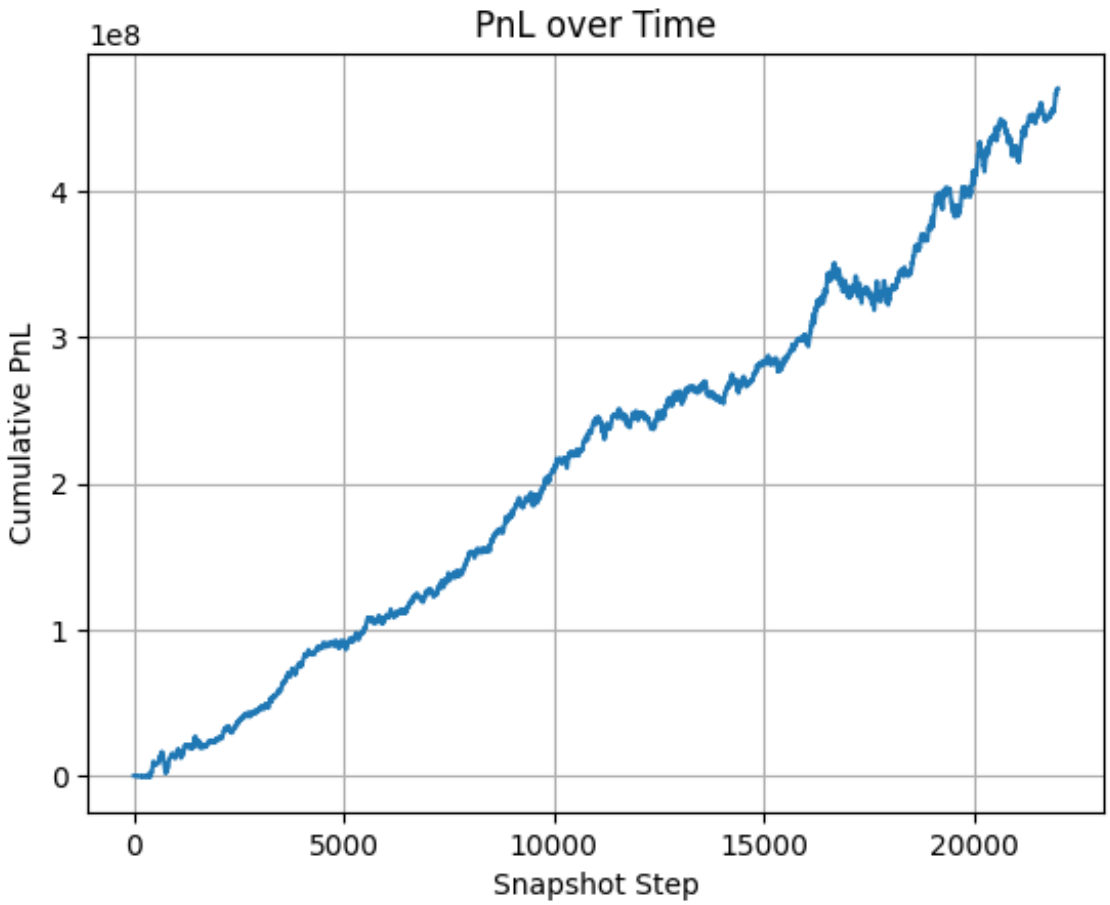
Bộ dữ liệu Test 7:



Hình 4.7: Biểu đồ PnL trên Bộ dữ liệu Test 7

Chỉ số Sharpe Ratio: 1.77493243

Bộ dữ liệu Test 8:



Hình 4.8: Biểu đồ PnL trên Bộ dữ liệu Test 8

Chỉ số Sharpe Ratio: 1.77669857

4.4 Nhận xét, giải thích kết quả, so sánh với các baseline

Dưới đây là bảng thống kê Sharpe Ratio thu được từ mô hình:

Bảng 4.1: Kết quả Sharpe Ratio của mô hình đề xuất trên từng tập dữ liệu

Dataset	Sharpe Ratio
Dataset 1	0.4848
Dataset 2	1.2243
Dataset 3	1.1454
Dataset 4	0.4403
Dataset 5	0.4052
Dataset 6	0.9806
Dataset 7	1.7749
Dataset 8	1.7767
Mean	**1.0290**
Variance	**0.2752**

Bảng dưới đây so sánh hiệu suất giữa các mô hình baseline và mô hình đề xuất dựa trên Sharpe Ratio trung bình và phương sai:

Mô hình	**Mean Sharpe Ratio**	**Variance**
Baseline 1 (RRL)	0.6000	0.0609
Baseline 2 (Ensemble RL)	0.2389	0.0711
Baseline 3 (RL + Indicators)	0.6561	0.3427
Proposed (RL + Game Theory)	**1.0290**	**0.2752**

Bảng 4.2: So sánh hiệu suất Sharpe Ratio giữa các mô hình

Bảng kết quả trên cho thấy mô hình đề xuất kết hợp Reinforcement Learning với Game Theory đạt hiệu suất vượt trội so với ba phương pháp baseline. Cụ thể, mô hình này đạt Sharpe Ratio trung bình là **1.0290**, cao hơn đáng kể so với các baseline còn lại.

Baseline 1 (RRL) đạt Sharpe Ratio trung bình **0.6000**, tuy là một trong những phương pháp cổ điển có hiệu quả nhất định, nhưng vẫn thấp hơn rõ rệt về mặt lợi nhuận điều chỉnh theo rủi ro. Baseline 2 (Ensemble RL), mặc dù áp dụng các thuật toán hiện đại như PPO, A2C và DDPG, lại chỉ đạt Sharpe Ratio trung bình **0.2389** cùng với phương sai tương đối cao (**0.0711**), phản ánh tính không ổn định của hiệu suất giữa các giai đoạn thị trường. Baseline 3 (RL kết hợp chỉ báo kỹ thuật) đạt kết quả trung bình cao hơn (**0.6561**), tuy nhiên phương sai cũng khá lớn (**0.3427**), cho thấy hiệu suất dao động nhiều, có thể do sự phụ thuộc vào chất lượng và độ trễ của các chỉ báo đầu vào.

Trong khi đó, mô hình đề xuất không chỉ đạt Sharpe Ratio cao nhất (**1.0290**) mà còn duy trì phương sai ở mức hợp lý (**0.2752**), cho thấy chiến lược học được không chỉ có hiệu suất tốt mà còn ổn định hơn qua các điều kiện thị trường khác nhau. Kết quả này phần nào củng cố giả thuyết rằng việc tích hợp yếu tố chiến lược từ Game Theory có thể giúp agent đưa ra các quyết định giao dịch hiệu quả hơn, khi xét đến bản chất tương tác giữa nhiều tác nhân trên thị trường thay vì chỉ tối ưu hóa cục bộ như các phương pháp baseline.

Tổng kết

Chương này đã trình bày và phân tích chi tiết hiệu quả của mô hình đề xuất kết hợp Reinforcement Learning với Game Theory, thông qua so sánh với ba phương pháp baseline phổ biến. Các kết quả thực nghiệm cho thấy mô hình đề xuất không chỉ đạt được Sharpe Ratio trung bình cao hơn rõ rệt mà còn duy trì được mức phương sai hợp lý, phản ánh tính ổn định trong hiệu suất giao dịch. Việc tích hợp Game Theory đã góp phần giúp agent đưa ra các quyết định phù hợp hơn trong môi trường thị trường có tính cạnh tranh và chiến lược cao. Tuy nhiên, một số hạn chế như sự biến

động của thị trường, tính bất định trong dữ liệu thực và giả định về tác động nhỏ của agent vẫn cần được tiếp tục xem xét trong các nghiên cứu tiếp theo. Những kết quả thu được là cơ sở vững chắc để tiếp tục mở rộng và phát triển mô hình trong các nghiên cứu trong tương lai.

KẾT LUẬN

Tổng kết

Đồ án này đã giải quyết một số vấn đề quan trọng trong bài toán giao dịch Market Making sử dụng học tăng cường, đặc biệt là trong bối cảnh thị trường tài chính thực có tính cạnh tranh và phi tuyến cao. Cụ thể, đồ án đã đóng góp ba điểm nổi bật:

- Thứ nhất, đồ án đề xuất một pipeline phân tích hành vi thị trường từ dữ liệu order book theo thời gian thực, bao gồm phân cụm bot, nhận diện chiến lược và xây dựng ma trận payoff.
- Thứ hai, đồ án tích hợp Lý thuyết Trò chơi (Game Theory) vào môi trường Reinforcement Learning thông qua reward được tính từ Nash Equilibrium giữa các chiến lược bot.
- Thứ ba, đồ án xây dựng môi trường Market Making dạng real-time RL với không gian quan sát mở rộng, cho phép agent học và thích nghi liên tục với thị trường biến động.

Mặt hạn chế và hướng phát triển trong tương lai

Tuy đã đạt được những kết quả tích cực về mặt mô phỏng và so sánh với các phương pháp baseline, đồ án vẫn còn một số hạn chế:

- Thứ nhất, mô hình mới chỉ được đánh giá trong môi trường mô phỏng, chưa được tích hợp và thử nghiệm trong hệ thống giao dịch thực tế (live-trading), do đó tính khả thi trong điều kiện thị trường trực tiếp vẫn chưa được kiểm chứng đầy đủ.
- Thứ hai, việc giả định rằng agent có tác động nhỏ đến thị trường giúp đơn giản hóa quá trình mô hình hóa, nhưng chưa phản ánh hết được các phản ứng phức tạp của thị trường khi khối lượng giao dịch lớn hơn được triển khai.

Trong tương lai, mô hình có thể được mở rộng để triển khai trong môi trường giao dịch thực (live-trading), tích hợp thêm thông tin vĩ mô hoặc dữ liệu phi cấu trúc nhằm cải thiện khả năng ra quyết định, mô phỏng hành vi động của các tác nhân thị trường, và tối ưu hiệu quả tính toán để đáp ứng yêu cầu của các hệ thống giao dịch tốc độ cao. Những hướng phát triển này sẽ giúp nâng cao tính ứng dụng và độ tin cậy của mô hình trong các môi trường tài chính thực tế.

REFERENCE

[1] Y. Nevmyvaka, Y. Feng **and** M. Kearns, *Reinforcement learning for optimized trade execution*, Available at `https://www.cis.upenn.edu/~mkearns/papers/rlexec.pdf`, 2006.

[2] A. S. Yasin **and** P. S. Gill, "Reinforcement learning framework for quantitative trading," *arXiv preprint arXiv:2411.07585*, 2024, Available at `https://doi.org/10.48550/arxiv.2411.07585`.

[3] S. Sarkar, "Quantitative trading using deep q learning," *International Journal for Research in Applied Science and Engineering Technology*, **jourvol** 11, **number** 4, **pages** 731–738, 2023. DOI: `10.22214/ijraset.2023.50170`. **url**: `https://doi.org/10.22214/ijraset.2023.50170`.

[4] J. Moody **and** M. Saffell, "Learning to trade via direct reinforcement," *IEEE Transactions on Neural Networks*, **jourvol** 12, **number** 4, **pages** 875–889, 2001. DOI: `10.1109/72.935097`. **url**: `https://doi.org/10.1109/72.935097`.

[5] M. Bertoluzzo **and** G. De Nicolao, "Reinforcement learning for optimal trading in stocks," *IEEE Transactions on Neural Networks*, **jourvol** 17, **number** 1, **pages** 212–222, 2006.

[6] L. Chen **and** Q. Gao, "Application of deep reinforcement learning on automated stock trading," **in***2019 10th International Conference on Software Engineering and Service Science (ICSESS)* IEEE, IEEE, 2019, **pages** 615–618. DOI: `10.1109/ICSESS47205.2019.9040728`. **url**: `https://doi.org/10.1109/ICSESS47205.2019.9040728`.

[7] X. Guo, X. Cheng **and** Y. Zhang, "Deep reinforcement learning for bitcoin trading," *IEEE Access*, **jourvol** 8, **pages** 169 069–169 076, 2020. DOI: `10.1109/ACCESS.2020.3023667`. **url**: `https://doi.org/10.1109/ACCESS.2020.3023667`.

[8] S. Gu, X. Wang, J. Chen **and** X. Dai, "Reinforcement learning for portfolio optimization in the presence of transaction costs," *Journal of Intelligent & Fuzzy Systems*, **jourvol** 41, **number** 3, **pages** 3853–3865, 2021.

[9] Y. Deng, F. Bao, Y. Kong, Z. Ren **and** Q. Dai, "Deep direct reinforcement learning for financial signal representation and trading," *IEEE Transactions on Neural Networks and Learning Systems*, **jourvol** 28, **number** 3, **pages** 653–664, 2017. DOI: `10.1109/TNNLS.2016.2522401`. **url**: `https://doi.org/10.1109/TNNLS.2016.2522401`.

[10] A. Azhikodan, A. Bhat **and** M. Jadhav, "Stock trading bot using deep reinforcement learning," **in***Innovations in Computer Science and Engineering* S. Satapathy, V. Bhateja **and** B. Janakiramaiah, **editors**, Springer, 2018, **pages** 41–49. DOI: 10.1007/978-981-10-8201-6_5. **url**: https://doi.org/10.1007/978-981-10-8201-6_5.

[11] Z. Jiang, D. Xu **and** J. Liang, *A deep reinforcement learning framework for the financial portfolio management problem*, 2017. arXiv: 1706.10059 [q-fin.PM]. **url**: https://arxiv.org/abs/1706.10059.

[12] J. C. Hull, *Options, Futures, and Other Derivatives*, 7th. Upper Saddle River, New Jersey: Pearson Prentice Hall, 2009.

[13] D. Fudenberg **and** J. Tirole, *Game Theory*. Cambridge, Massachusetts: MIT Press, 1991.

[14] S. Kakutani, "A generalization of brouwer's fixed point theorem," *Duke Mathematical Journal*, **jourvol** 8, **number** 3, **pages** 457–459, 1941. DOI: 10.1215/S0012-7094-41-00837-7. **url**: http://dml.mathdoc.fr/item/1077492791/.

[15] J. F. Nash, "Equilibrium points in n-person games," *Proceedings of the National Academy of Sciences*, **jourvol** 36, **number** 1, **pages** 48–49, 1950. DOI: 10.1073/pnas.36.1.48. **url**: https://doi.org/10.1073/pnas.36.1.48.

[16] I. L. Glicksberg, "A further generalization of the kakutani fixed-point theorem, with applications to nash equilibrium points," RAND Corporation, techreport P-193, 1951, Available online at RAND.org. **url**: https://www.rand.org/pubs/papers/P193.html.

[17] T. P. Lillicrap, J. J. Hunt, A. Pritzel **andothers**, *Continuous control with deep reinforcement learning*, 2015. arXiv: 1509.02971 [cs.LG]. **url**: https://arxiv.org/abs/1509.02971.